[BUSINESS] FORECASTING AND PREDICTIVE MODELING

LECTURE 5

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

In Lecture 4 ...

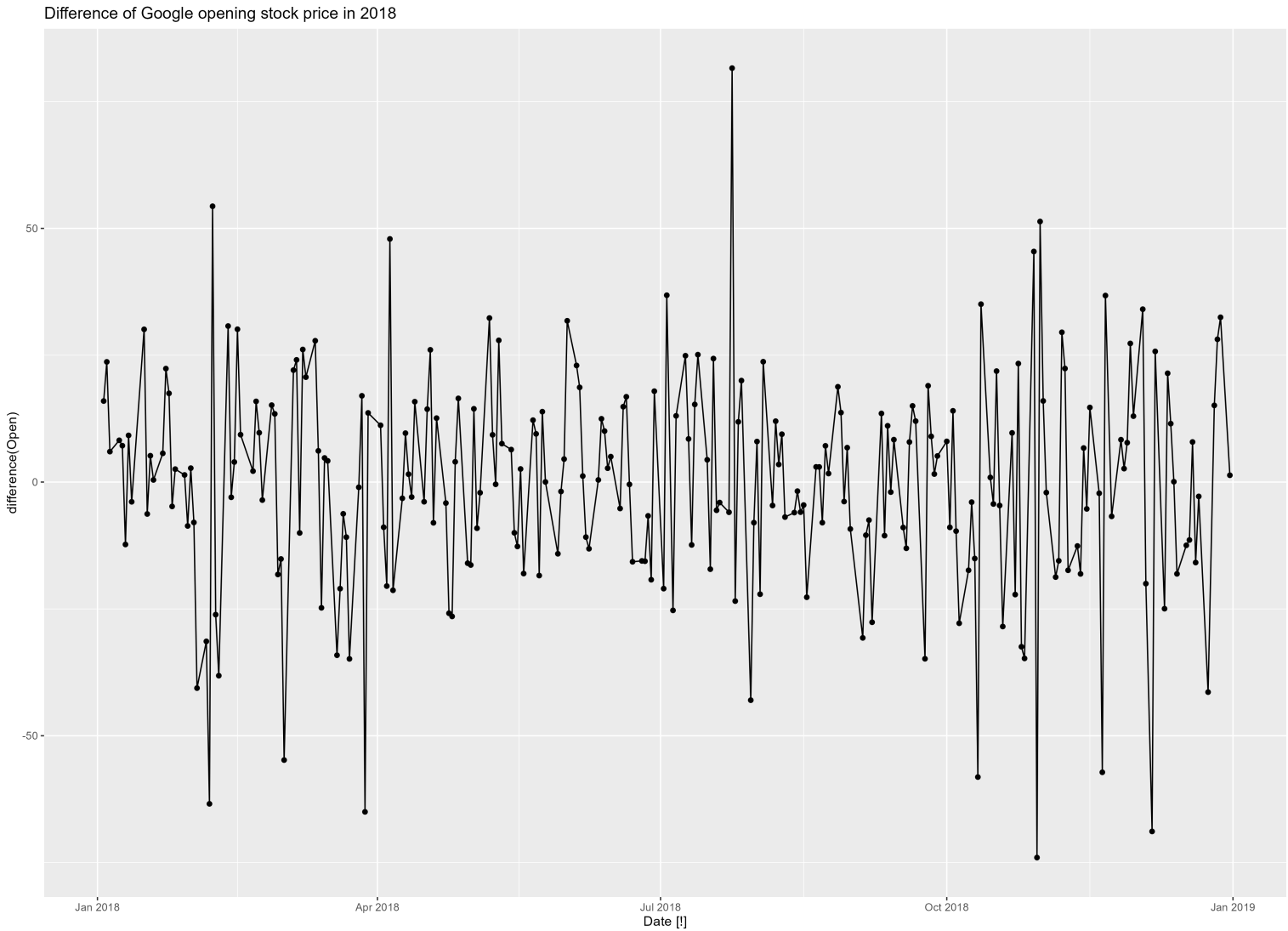
- Autocorrelation plot
- Stationarity
- Differencing

Today

- Time series transformations
- Time series decomposition

Stationary time series - recap

- A time series is *stationary* if its observations are not dependent on time and summary statistics computed for it (ex. mean and variance) are not dependent on time.
- In other words, a time series is stationary, if
  - it has no trend (it is roughly horizontal)
  - it has constant variance (it is homoscedastic)
  - it has no predictable patterns (no seasons).
- **Obs.** white noise is stationary (but not every stationary time series is white noise).
- Some forecasting methods require time series to be stationary.


Difference of Google opening stock price in 2018
50
0
-50
difference(Open)
Jan 2018
Apr 2018
Jul 2018
Oct 2018
Jan 2019
Date [!]

- If we have identified that our time series is not stationary, and we need a stationary time series, we can do the following:
  - Use a transformation to stabilize the variance
  - Use differencing to stabilize the mean - this was discussed in Lecture 4.

Time series transformations

- Before decomposing a time series or training a forecasting model it might be useful to perform some form of adjustment on the data.
- These adjustments can help simplify the data, remove variation or the effect of other factors.
- The simpler the time series, the easier to model it and the more accurate the resulting forecast.

Calendar adjustment I

- Calendar adjustment is the method for removing the *calendar effect* from a time series.
- Calendar effect can depend on:
  - the number of working days (in general between 18 and 23 in a month) or simply the number of days in a month
  - the number of trading days in a month (more people shop on Saturday than on Monday, so months with 5 Saturdays have higher retail shopping values)
  - the timing of certain public holidays (ex. Easter, which may be in April or May or partially in both or the Chinese New Year which is in general in February, but can be in January)
  - the occurrence of a leap year
  - the fact that some public holidays might overlap with non-working days

Calendar adjustment II

- *Working day adjustment* focuses on the changing number of working days. This can be eliminated by simply dividing the monthly data by the number of working days, to get an average value.

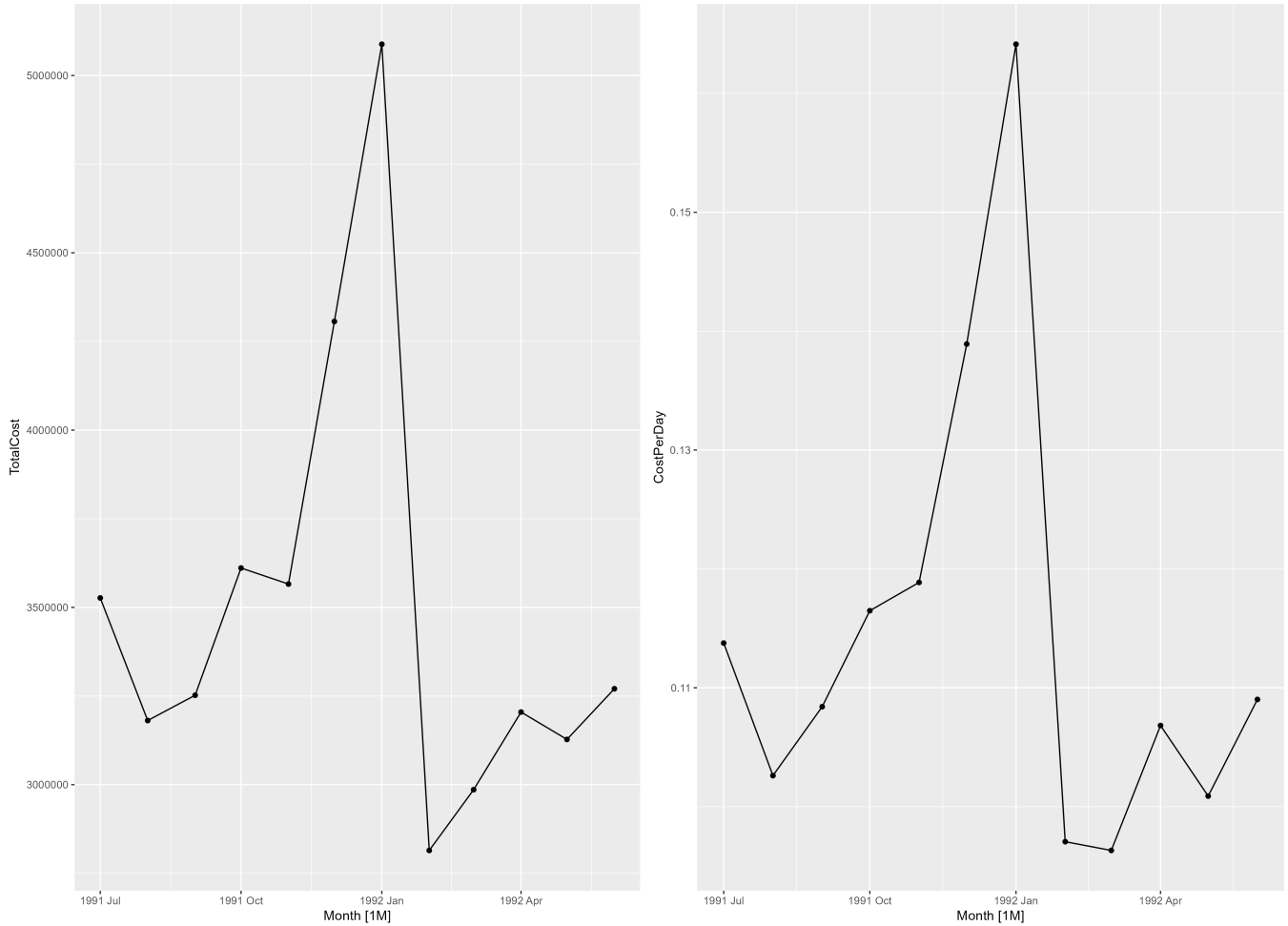
- Other calendar effects might be more complicated to remove.

- One reason why time series decomposition is performed, is to do *seasonal adjustment*: remove the seasonal effects from the data (in many cases we are rather interested in the trend, not the seasonal variations).
- This includes removing the seasonal component from the data, but it is often combined with calendar adjustments as well.

- EuroStat's Guidelines on seasonal adjustment can be found here `https://ec.europa.eu/eurostat/documents/3859598/6830795/KS-GQ-15-001-EN-N.pdf`, while the full Handbook on Seasonal Adjustment (2018 edition) can be found here: `https://ec.europa.eu/eurostat/documents/3859598/8939616/KS-GQ-18-001-EN-N.pdf/7c4d120a-4b8a-441b-aefd-6afe81a7cf59?t=1533194231000`

Calendar adjustment example

```
a10 <- PBS |>
  filter(ATC2 == "A10") |>
  select(Month, Concession, Type, Cost) |>
  summarize(TotalCost = sum(Cost)) |>
  mutate(Cost = TotalCost / 1000000)

a10_one_year <- a10 |> head(n = 12)

#save plot in variable to display two plots on plot_grid
p1 <- a10_one_year |>
  autoplot() +
  geom_point()

p1

a10_one_year <- a10_one_year |>
  mutate(CostPerDay = Cost / days_in_month(Month))

p2 <- a10_one_year |>
  autoplot(CostPerDay) +
  geom_point()

p2

plot_grid(p1, p2)
```

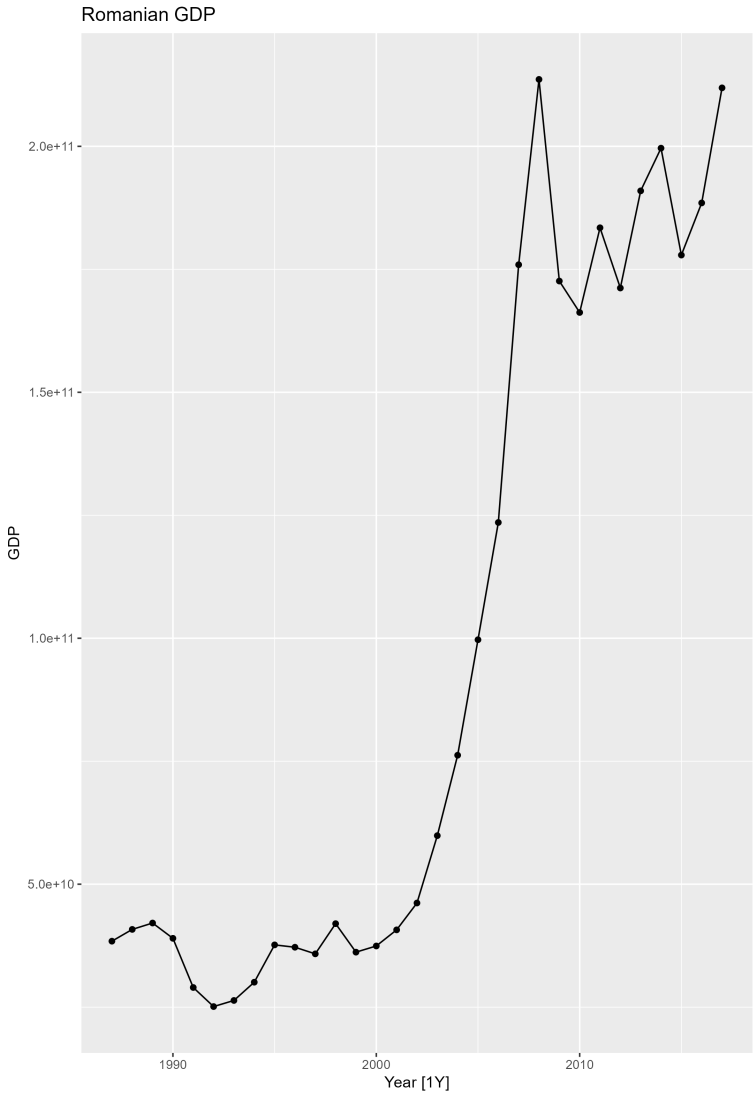


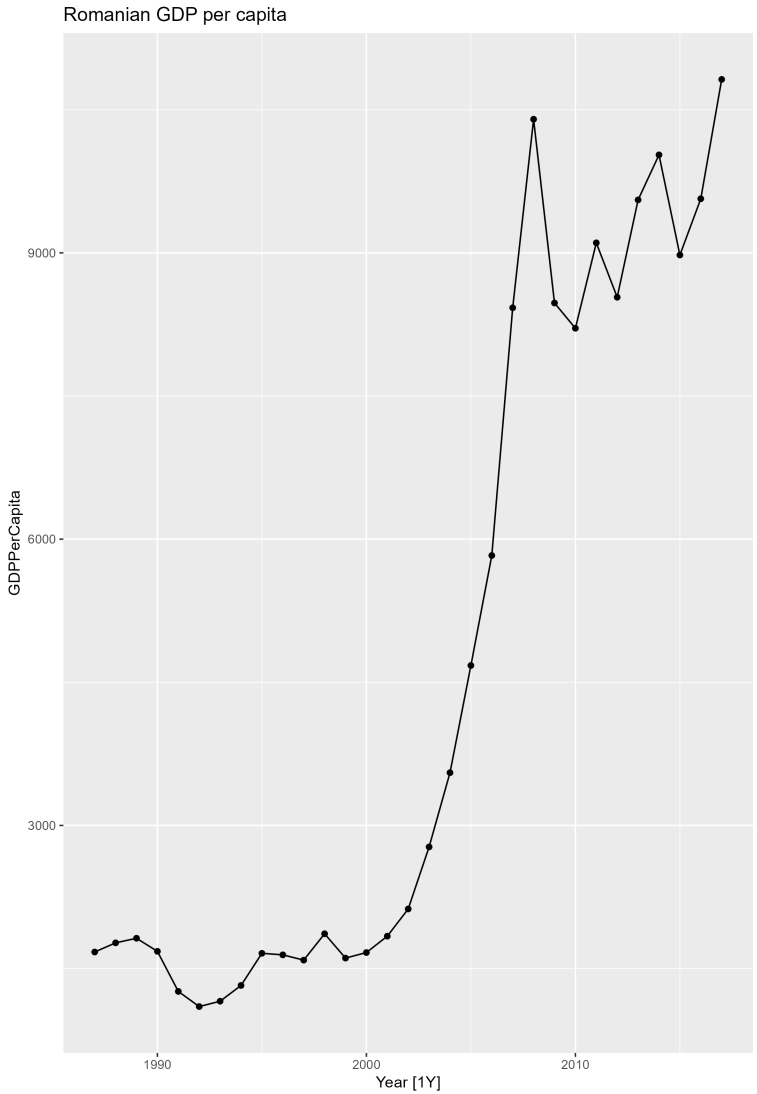
- What do you notice on the two plots?

Population adjustment

- Data that can be affected by population changes should be adjusted to provide per-capita values (or per thousand, million, etc. persons).
- For example, an increase in the number of hospital beds, might be simply caused by an increase in the population, not neccessarily an improvement in the health system.

```
ro_economy <- global_economy |>
  filter(Country == "Romania", is.na(GDP) == FALSE, is.na(Population) == FALSE)
ro_economy |>
  autoplot(GDP) +
  geom_point() +
  labs(title = "Romanian GDP") -> p1
p1

ro_economy |>
  mutate(GDPPerCapita = GDP / Population) -> ro_economy

ro_economy |> autoplot(GDPPerCapita) +
  geom_point() +
  labs(title = "Romanian GDP per capita") -> p2
p2

plot_grid(p1, p2)
```


Romanian GDP
GDP
2.0e+11
1.5e+11
1.0e+11
5.0e+10
1990
2000
2010
Year [1Y]


Romanian GDP per capita
GDPPerCapita
9000
6000
3000
1990
2000
2010
Year [1Y]

Inflation adjustment

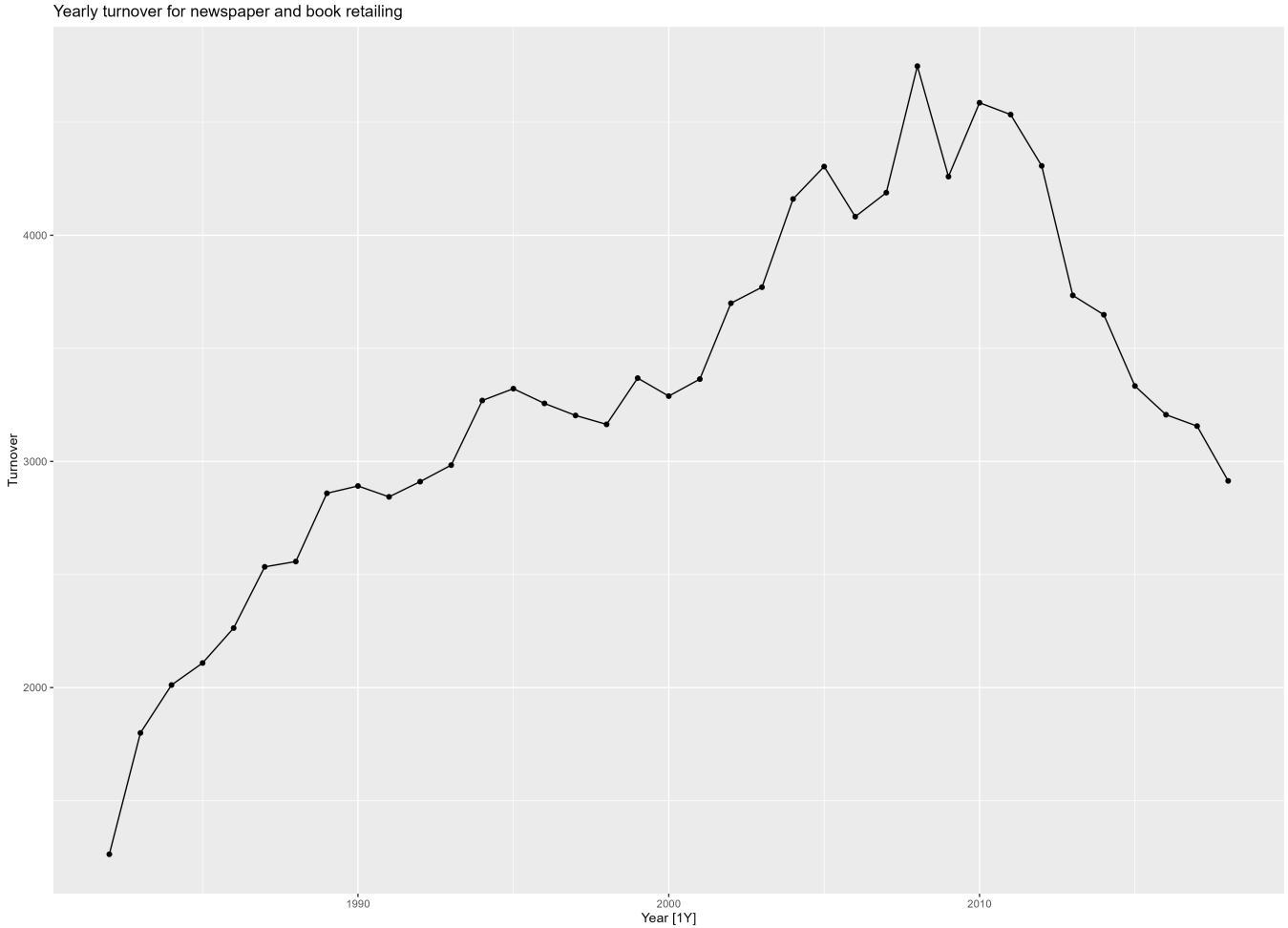
- Data that is affected by the value of the money also has to be adjusted: the price of a house today cannot be directly compared to the price of a house 10 years ago.
- This is why such time series are adjusted so that all values are reported according to the values for a specific year.
- For this, a so-called *price-index* is used. For example, for the price of goods, the *Consumer Price Index (CPI)* is used.
- If we denote the price index for year t by z_t, the original value for year t by y_t and we want the adjusted price, x_t for 2023, we can compute it by:

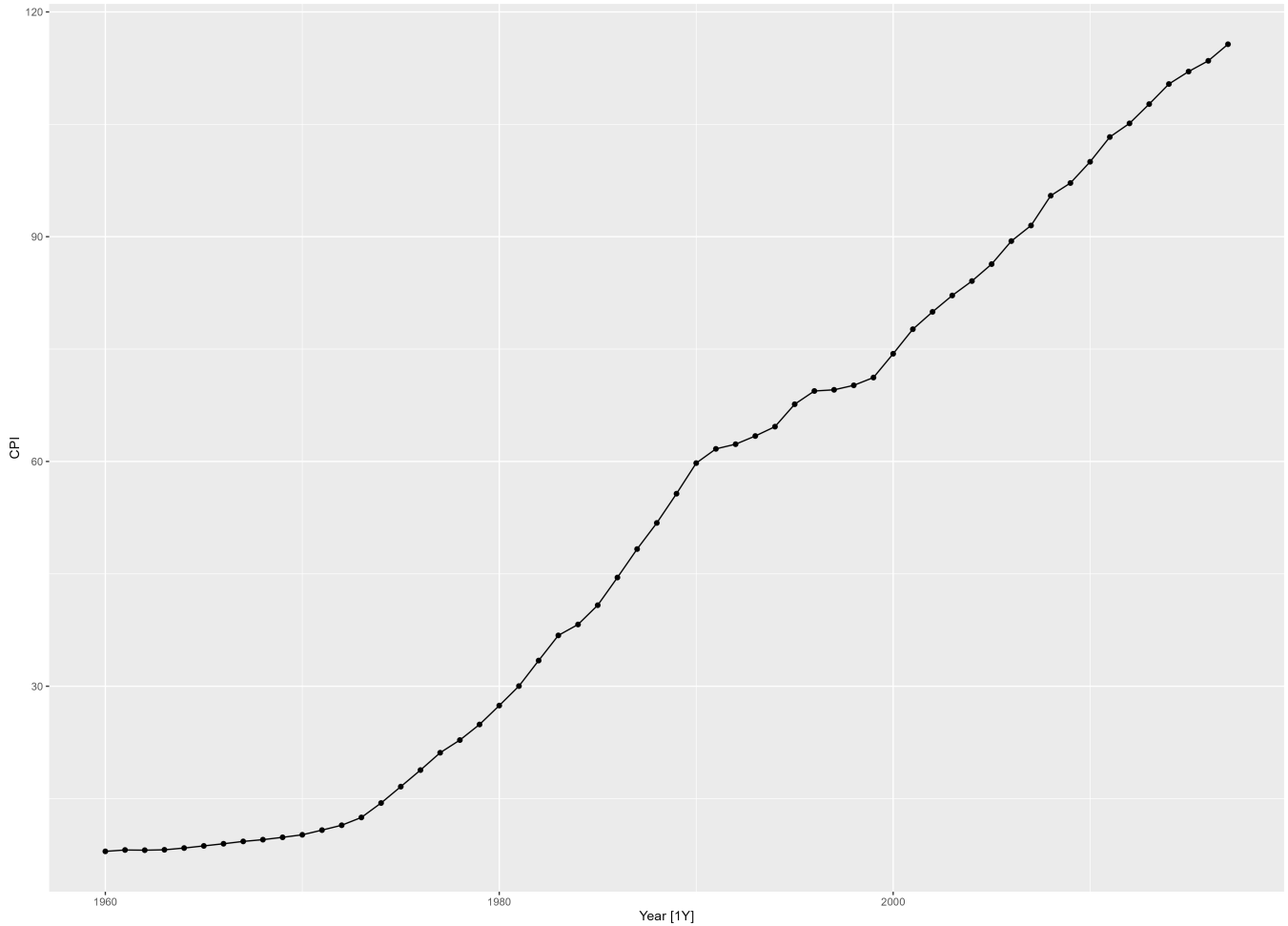
$$x_t = \frac{y_t}{z_t} * z_{2023}$$

Example of inflation adjustment

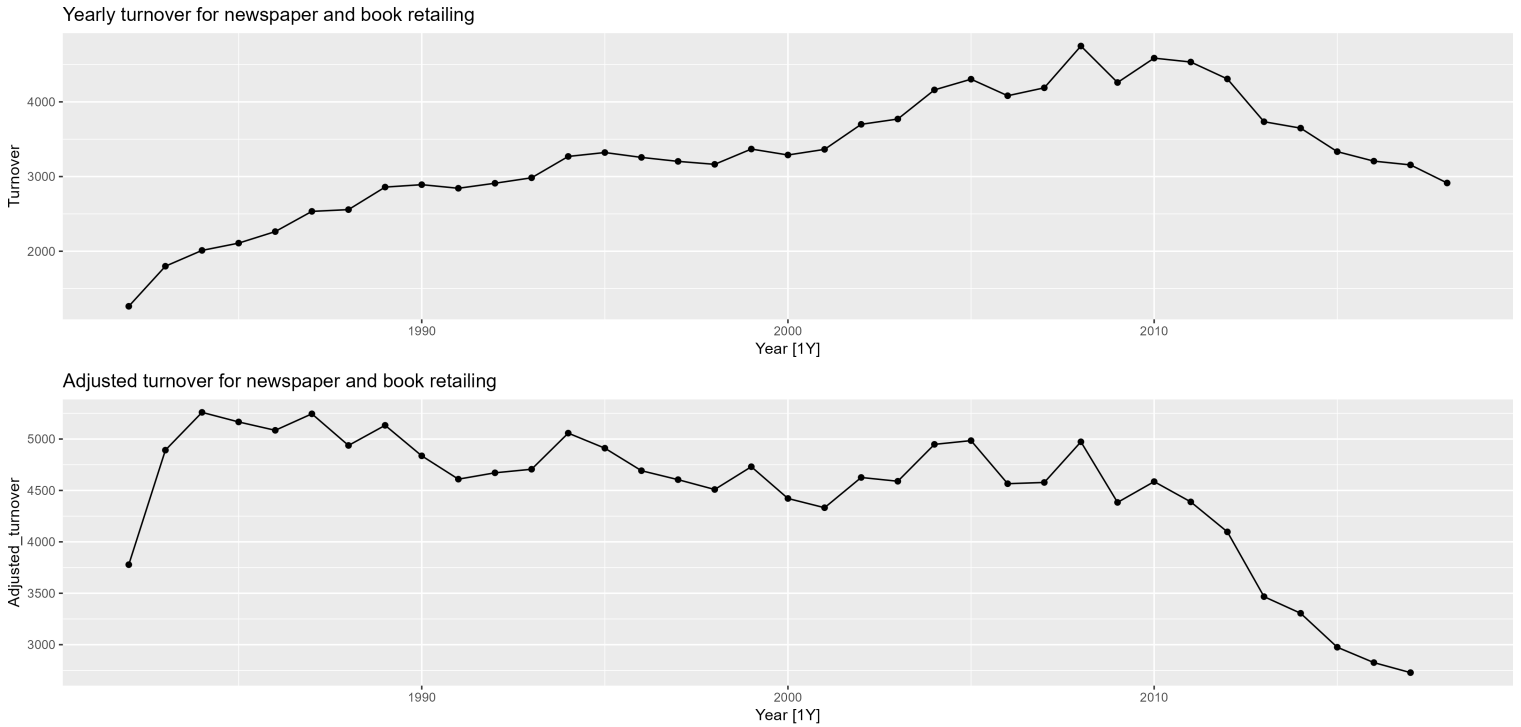
- The *global_economy* time series contains the CPI for each country and year.
- We can use it to see Australia's newspaper and book industry retail over the years.

```r
book_retail <- aus_retail |>
  filter(Industry == "Newspaper and book retailing")

book_retail |>
  group_by(Industry) |>
  index_by(Year = year(Month)) |>
  summarize(Turnover = sum(Turnover)) -> yearly_book_retail

p1 <- yearly_book_retail |>
  autoplot(Turnover) +
  geom_point() +
  labs(title = "Yearly turnover for newspaper and book retailing")

aus_economy <- global_economy |>
  filter(Country == "Australia")

aus_economy |>
  autoplot(CPI) +
  geom_point() +
  labs("CPI for Australia")
ggsave("TimeSeriesPlot_Australian_CPI.png")

yearly_book_retail_CPI <- yearly_book_retail |>
  left_join(aus_economy, by = "Year")

yearly_book_retail_CPI |>
  mutate(Adjusted_turnover = Turnover / CPI * 100) |>
  select(Year, Turnover, Adjusted_turnover) -> book_retail_CPI

p2 <- book_retail_CPI |>
  autoplot(Adjusted_turnover) +
  geom_point() +
  labs(title = "Adjusted turnover for newspaper and book retailing")

plot_grid(p1, p2, nrows = 2, ncol = 1)
```


Yearly turnover for newspaper and book retailing
Turnover
4000
3000
2000
1990
2000
2010
Year [1Y]


120
90
60
30
CPI
1960
1980
2000
Year [1Y]

- Comparison between turnover and CPI adjusted turnover for Newspaper and book retailing industry

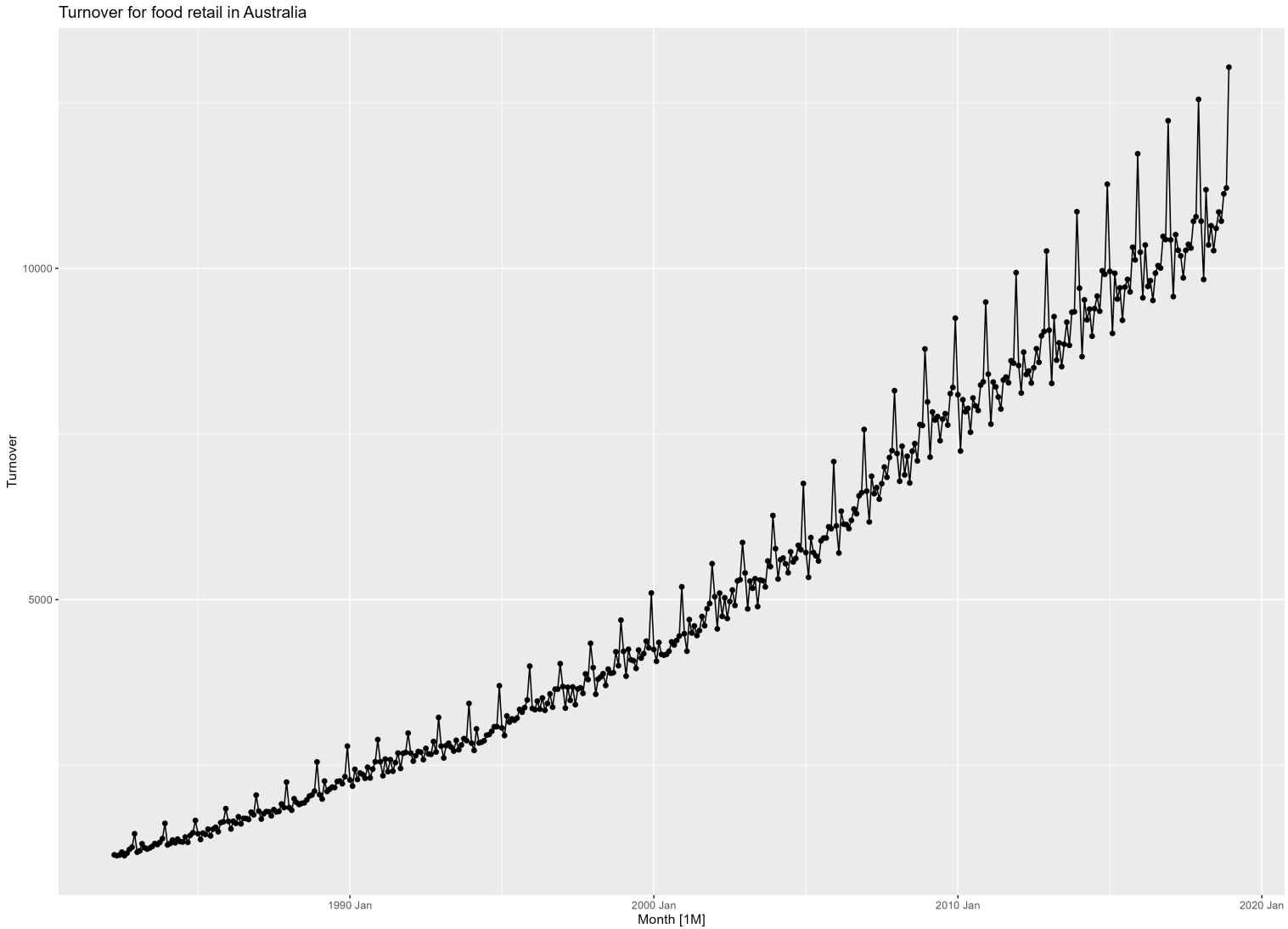


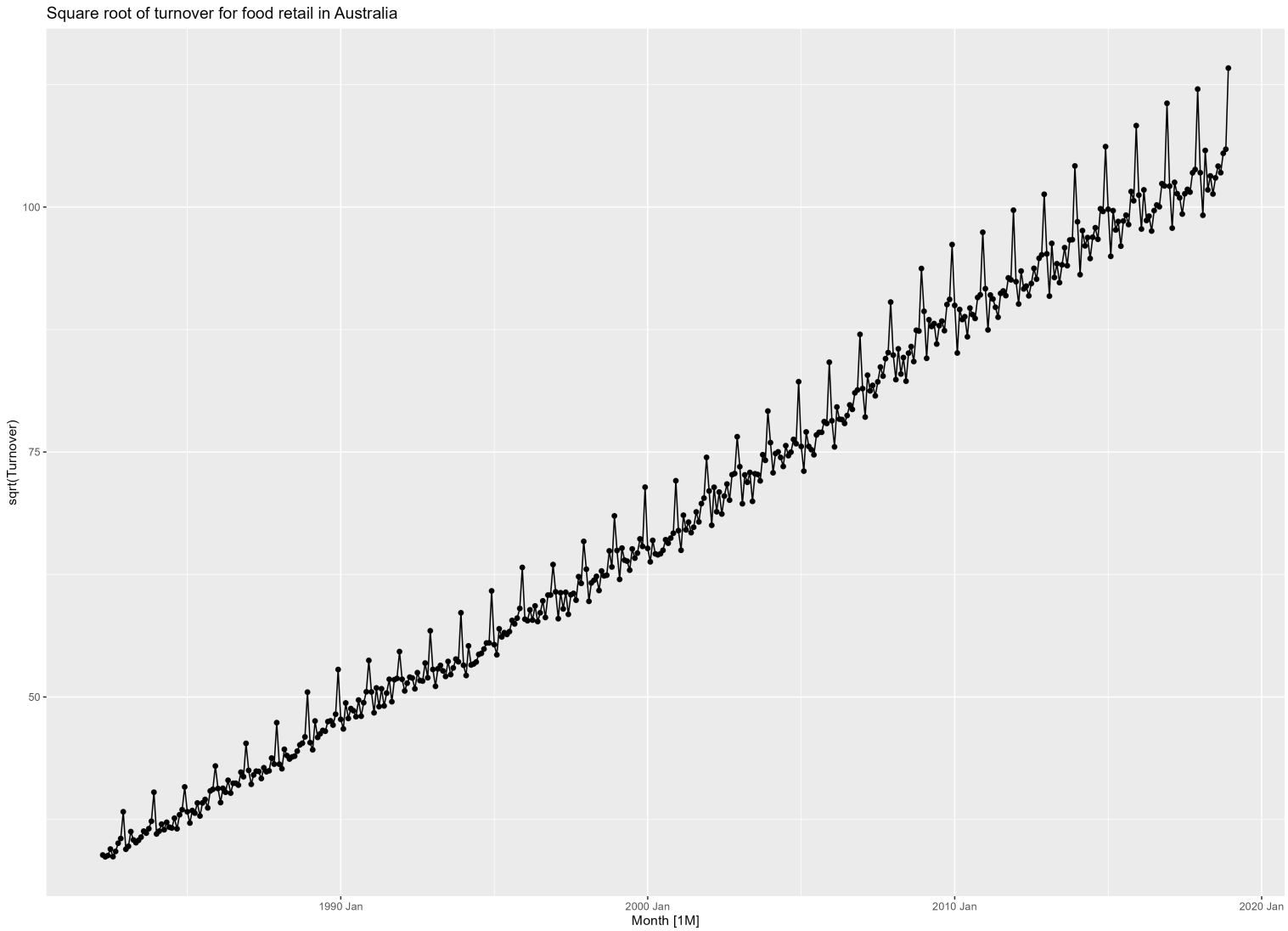
Mathematical transformations

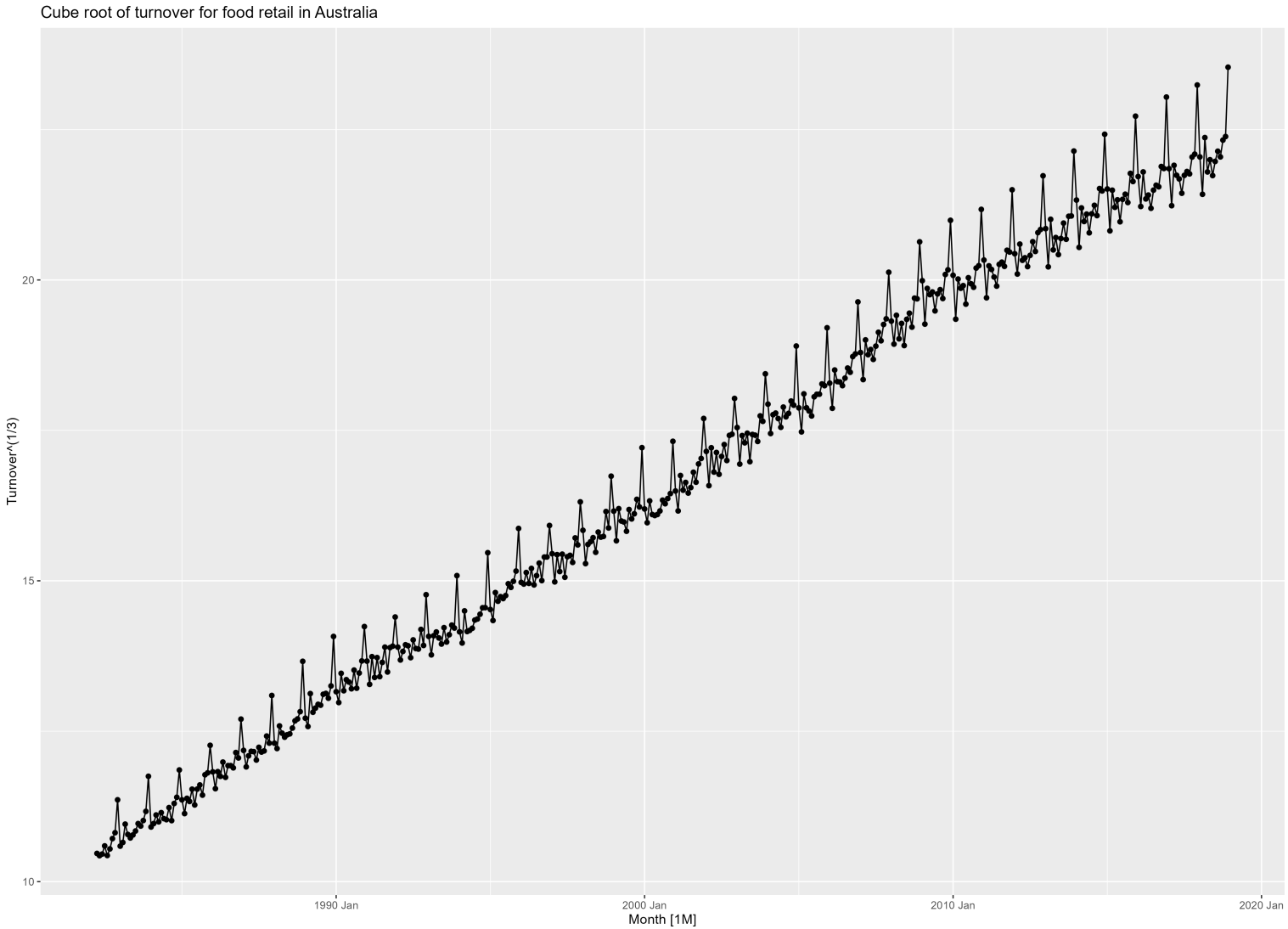
- These are transformations to remove the change (increase and decrease) in the variation of the data.
- The most frequently used mathematical transformations are:
  - Square root
  - Cube root
  - Take the logarithm
- The above order is the order of strength for these transformations.
- Logarithms are the most interpretable: a change in the log value is percentage change in the original data.

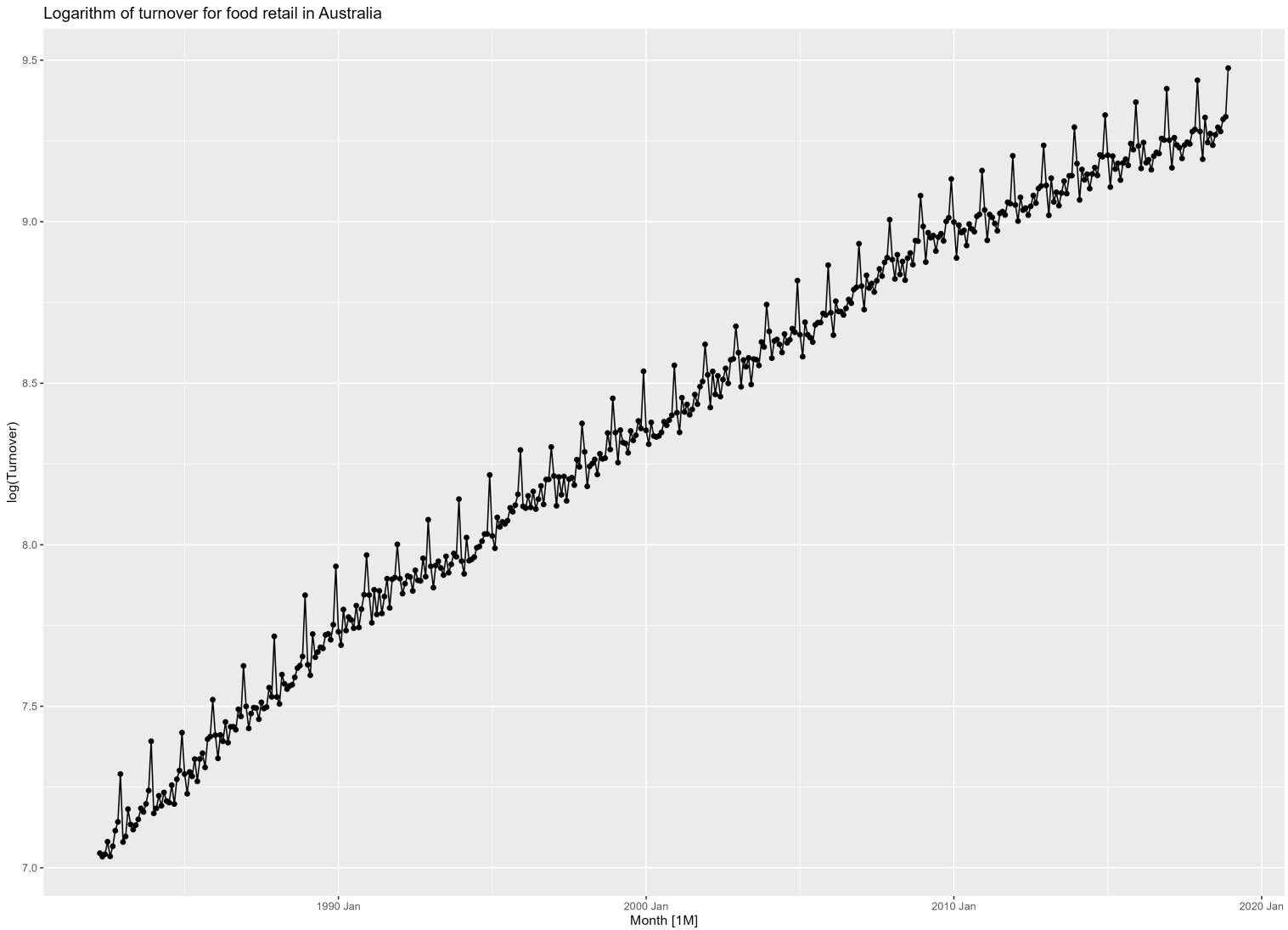
Example

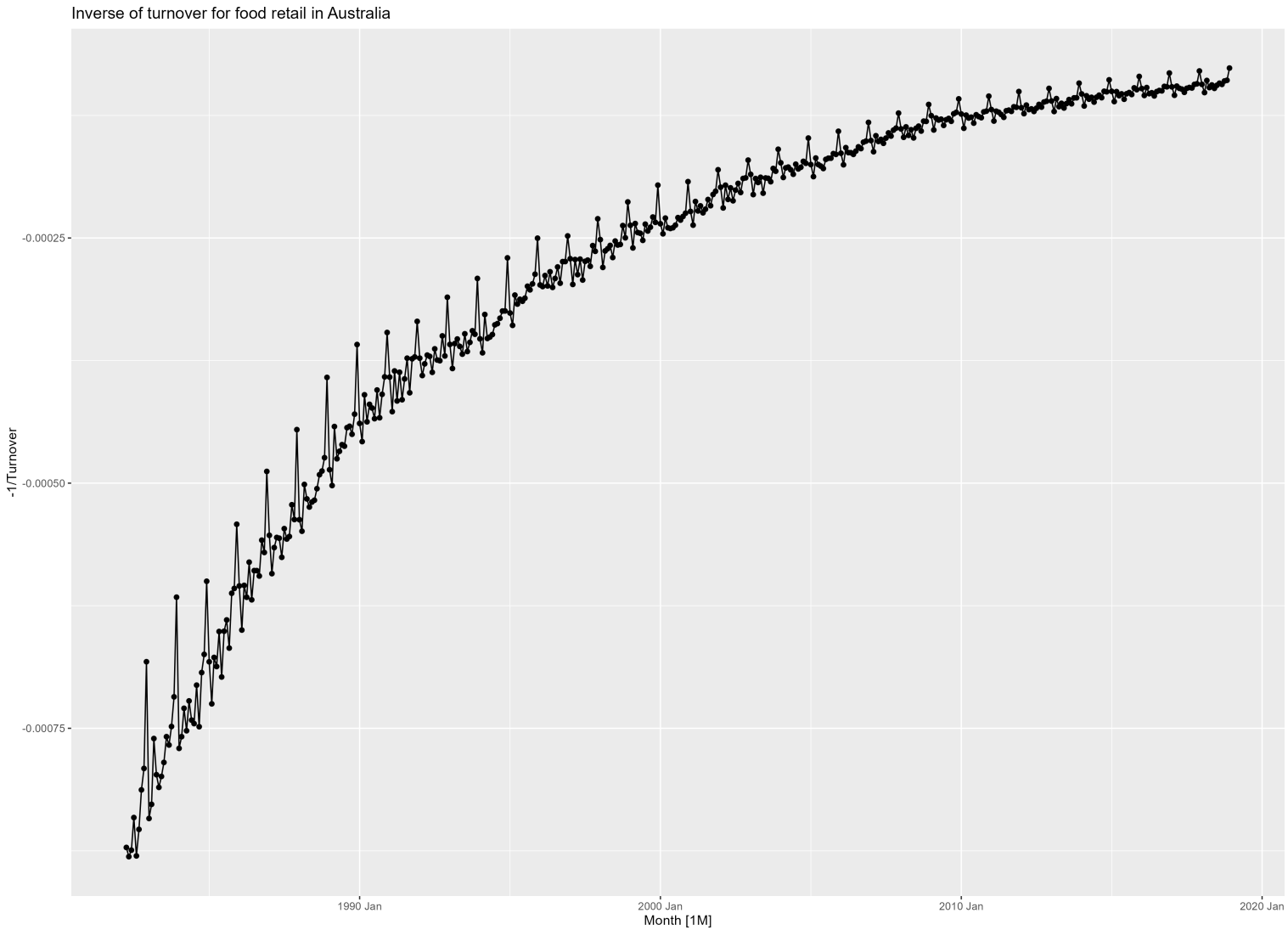
```
food_retail <- aus_retail |>
  filter(Industry == "Food retailing") |>
  summarize(Turnover = sum(Turnover))

food_retail |>
  autoplot() +
  geom_point() +
  labs(title = "Turnover for food retail in Australia")

food_retail |>
  autoplot(sqrt(Turnover)) +
  geom_point() +
  labs(title = "Square root of turnover for food retail in Australia")

food_retail |>
  autoplot(Turnover^(1/3)) +
  geom_point() +
  labs(title = "Cube root of turnover for food retail in Australia")

food_retail |>
  autoplot(log(Turnover)) +
  geom_point() +
  labs(title = "Logarithm of turnover for food retail in Australia")

food_retail |>
  autoplot(-1/Turnover) +
  geom_point() +
  labs(title = "Inverse of turnover for food retail in Australia")
```


Turnover for food retail in Australia
10000
5000
Turnover
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month [1M]


Square root of turnover for food retail in Australia
sqrt(Turnover)
100
75
50
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month [1M]


Cube root of turnover for food retail in Australia
Turnover^(1/3)
20
15
10
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month [1M]


Logarithm of turnover for food retail in Australia
9.5
9.0
8.5
8.0
7.5
7.0
log(Turnover)
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month [1M]


Inverse of turnover for food retail in Australia
-0.00025
-0.00050
-0.00075
-1/Turnover
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month [1M]

Box-Cox transformations I

- Notation: the original time series is denoted by $y_1 .... y_t$ and that the transformed time series will be denoted by $w_1 .... w_t$.
- It is a family of transformations, which includes both the logarithmic and the power transformations, through a parameter λ.

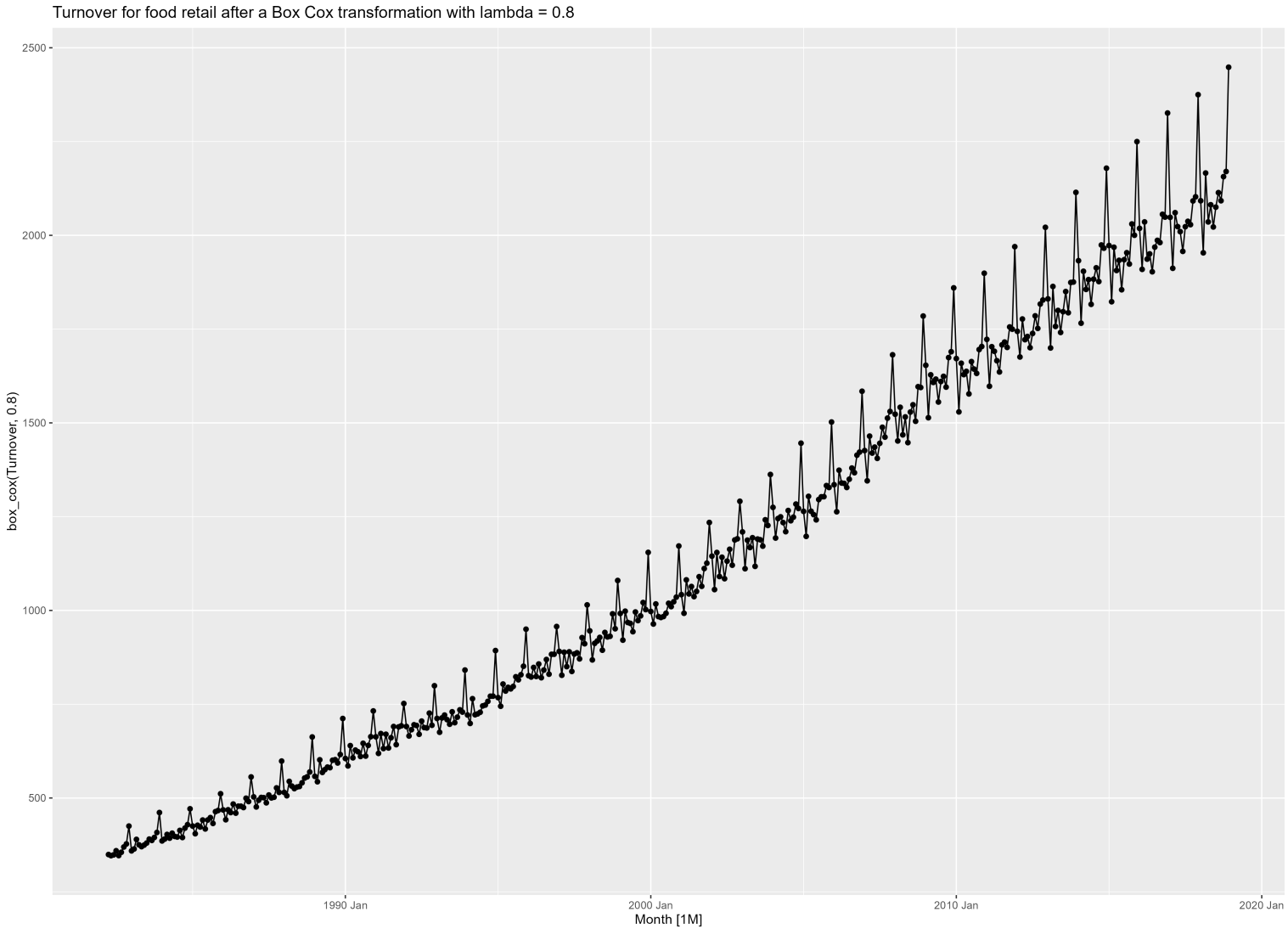
$$w_t = \begin{cases} log(y_t), & \text{if } \lambda = 0 \\ \frac{sign(y_t) * |y_t|^{\lambda} - 1}{\lambda} & \text{otherwise} \end{cases}$$

- Actually, this is a modified version of the Box-Cox transformation which allows for negative values in y_t (assuming that λ is not 0).

Box-Cox transformations II

- If $\lambda = 0$, we have the logarithmic transformation (natural logarithm).
- If $\lambda = 0.5$, we have the square root transformation.
- If $\lambda = 1$, $w_t = y_t - 1$ (lower values, but exactly the same shape for the series). For other values, the shape of the series will change.
- If $\lambda = -1$, we have the reciprocal transformation and for -0.5 we have the reciprocal square root transformation.
- In R, we can compute the Box Cox transformation using the *box_cox* function, which takes as parameter the data and the value of λ.

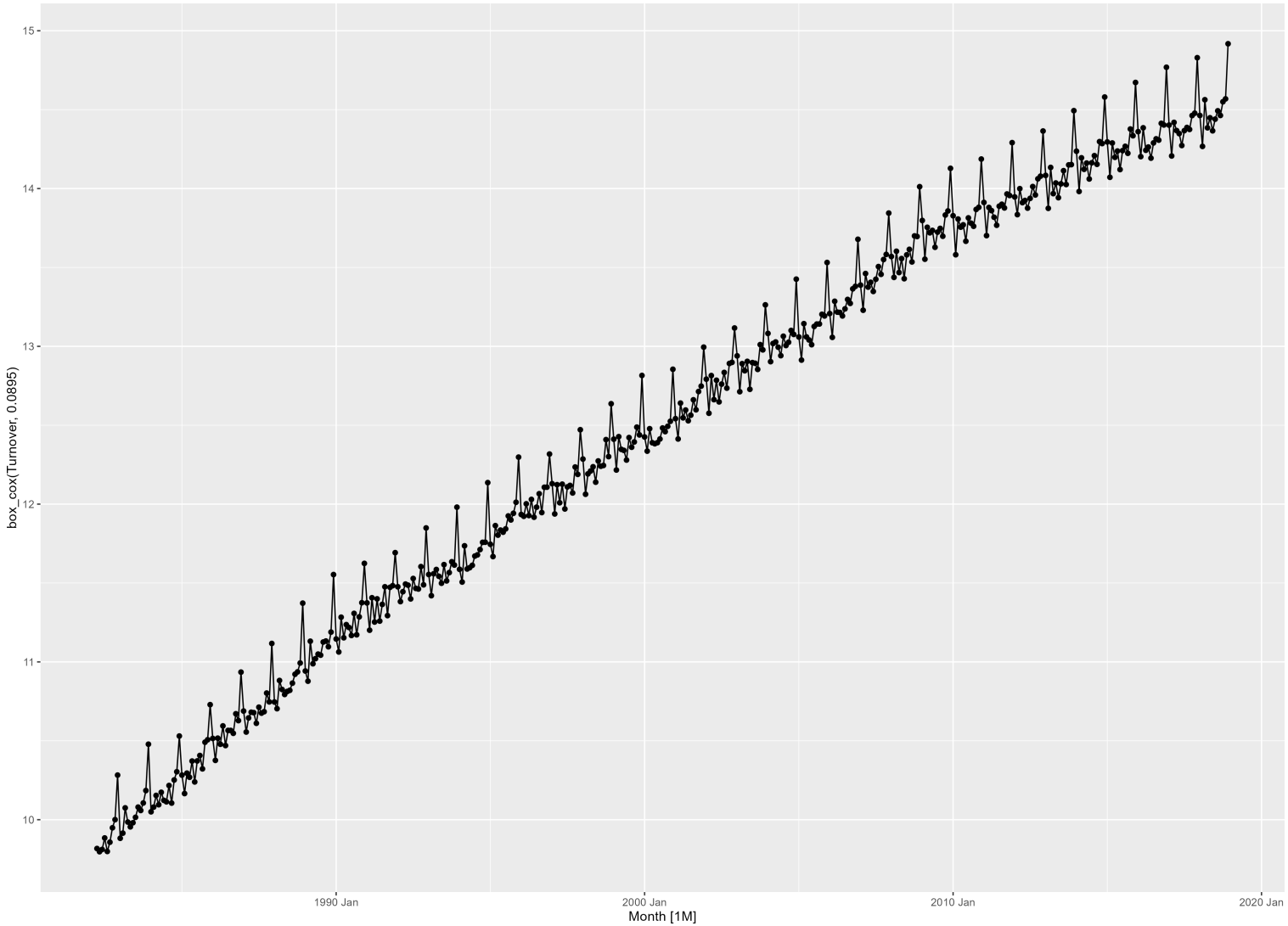
```
food_retail |>
    autoplot(box_cox(Turnover, 0.8)) + geom_point()
```


Turnover for food retail after a Box Cox transformation with lambda = 0.8
box_cox(Turnover, 0.8)
2500
2000
1500
1000
500
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month [1M]

- The goal is to find a value for λ for which the seasonal variation is the same across the entire time series. This might be complicated, since you need to try different values until you find a good one.
- An alternative is to use the *guerrero* feature, which finds the right value for λ automatically.

```
food_retail |> features(Turnover, guerrero)
```

```
+   features(Turnover, guerrero)
# A tibble: 1 x 1
  lambda_guerrero
            <dbl>
1          0.0895
>
```


15
14
13
12
11
10
box_cox(Turnover, 0.0895)
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month [1M]

Time series decomposition

- We have discussed that in case of a time series we can identify a *trend*, *season* and *cycle*. When we talk about decomposing a time series, we consider the cycle and trend together, in a component called *trend-cycle* (or simply just trend).

- Consequently, a time series y_t can be considered as being made of three components:
  - trend (or trend-cycle) - T,
  - seasonality - S
  - remainder - R.

- We can assume an additive decomposition and consider that:

$$y_t = S_t + T_t + R_t$$

where y are the data, S is the seasonal component, T is the trend component, and R is the remainder, all in period t.

- Alternatively, we can assume a multiplicative decomposition, and consider that:

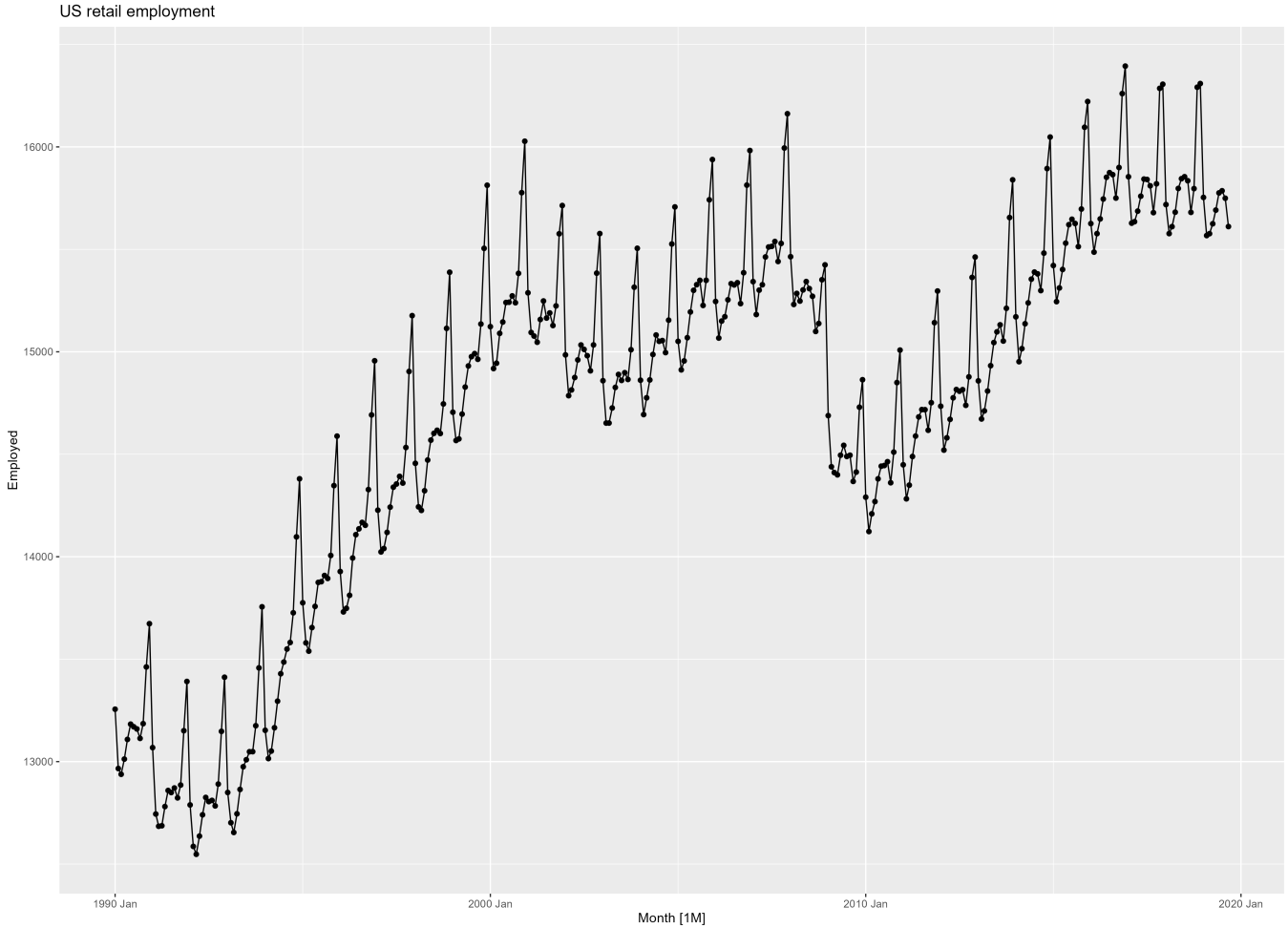
$$y_t = S_t * T_t * R_t$$

- Multiplicative decomposition is a better alternative when the seasonal or cyclic fluctuations vary with the level of the time series (common in case of economic series). Alternatively, we can use a Box-Cox transformation on it, and use an additive decomposition.
- When there is no such variation, additive decomposition is appropriate.

Examples

- Let us look at a first example of US retail trade unemployment rates.

```
us_retail_employment <- us_employment |>
  filter(Title == "Retail Trade", year(Month) >= 1990) |>
  select(-Series_ID, -Title)

us_retail_employment
us_retail_employment |>
  autoplot() +
  geom_point() +
  labs(title = "US retail employment")
```


US retail employment
16000
15000
14000
13000
Employed
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month [1M]

Decomposition example

- We will talk about how to the decomposition is done, but let's see first some examples of how a decomposition looks like.
- One function to decompose a time series is the *STL* function. It can have many parameters to customize the decomposition, but currently, we will only focus on the default values (so no parameters passed).
- On the other hand, STL can only do additive decomposition.
- Let's see how we can decompose the US retail unemployment data.

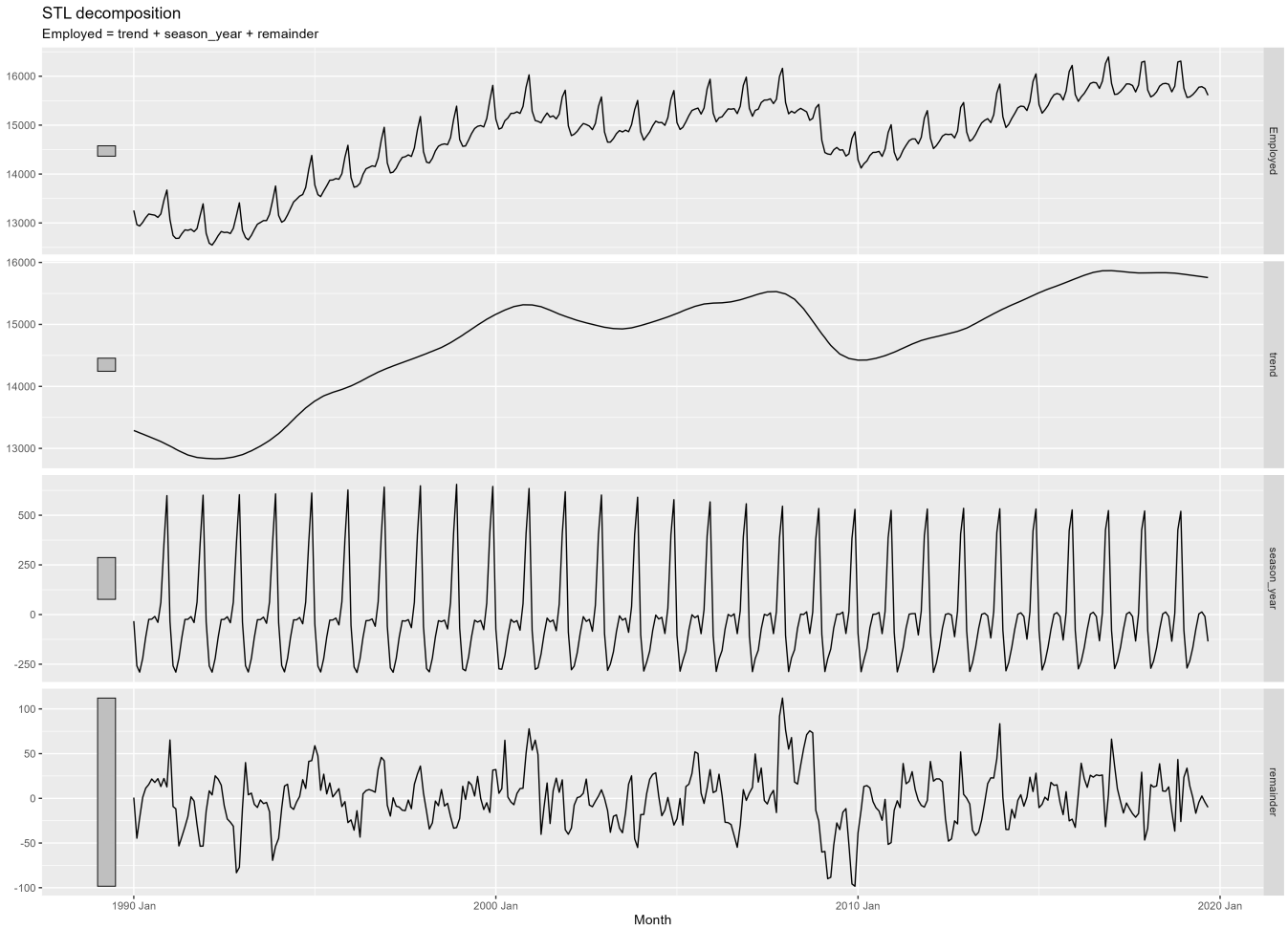
```
dcmp <- us_retail_employment |>
  model(stl = STL(Employed))
dcmp
components(dcmp)
```

- We can see that the result of the decomposition is a *mable*, a *model table*.
- Calling the *components* function on *dcmp* we will get a *dabble*, a *decomposition table*, which contains both the initial data and the three components in which it was separated. The third comment line shows that indeed an additive decomposition was performed.
- Column *Month* is the timestamp of the observation.
- Column *Employed* contains the initial values for the Employed attribute for each observation.
- Column *trend* shows the trend component from the decomposition for each observation.
- Column *season_year* shows the seasonal component from the decomposition for each observation.
- Column *remainder* shows the remainder component from the decomposition for each observation.

- Components of the decomposition

```
> components(dcmp)
# A dable: 357 x 7 [1M]
# Key:     .model [1]
# :        Employed = trend + season_year + remainder
   .model    Month Employed  trend season_year remainder season_adjust
   <chr>     <mth>    <dbl>  <dbl>       <dbl>     <dbl>          <dbl>
 1 stl    1990 Jan   13256. 13288.      -33.0      0.836         13289.
 2 stl    1990 Feb   12966. 13269.     -258.      -44.6           13224.
 3 stl    1990 Mar   12938. 13250.     -290.      -22.1           13228.
 4 stl    1990 Apr   13012. 13231.     -220.       1.05           13232.
 5 stl    1990 May   13108. 13211.     -114.      11.3            13223.
 6 stl    1990 Jun   13183. 13192.      -24.3     15.5            13207.
 7 stl    1990 Jul   13170. 13172.      -23.2     21.6            13193.
 8 stl    1990 Aug   13160. 13151.       -9.52    17.8            13169.
 9 stl    1990 Sep   13113. 13131.      -39.5      22.0           13153.
10 stl    1990 Oct   13185. 13110.       61.6      13.2           13124.
# i 347 more rows
```

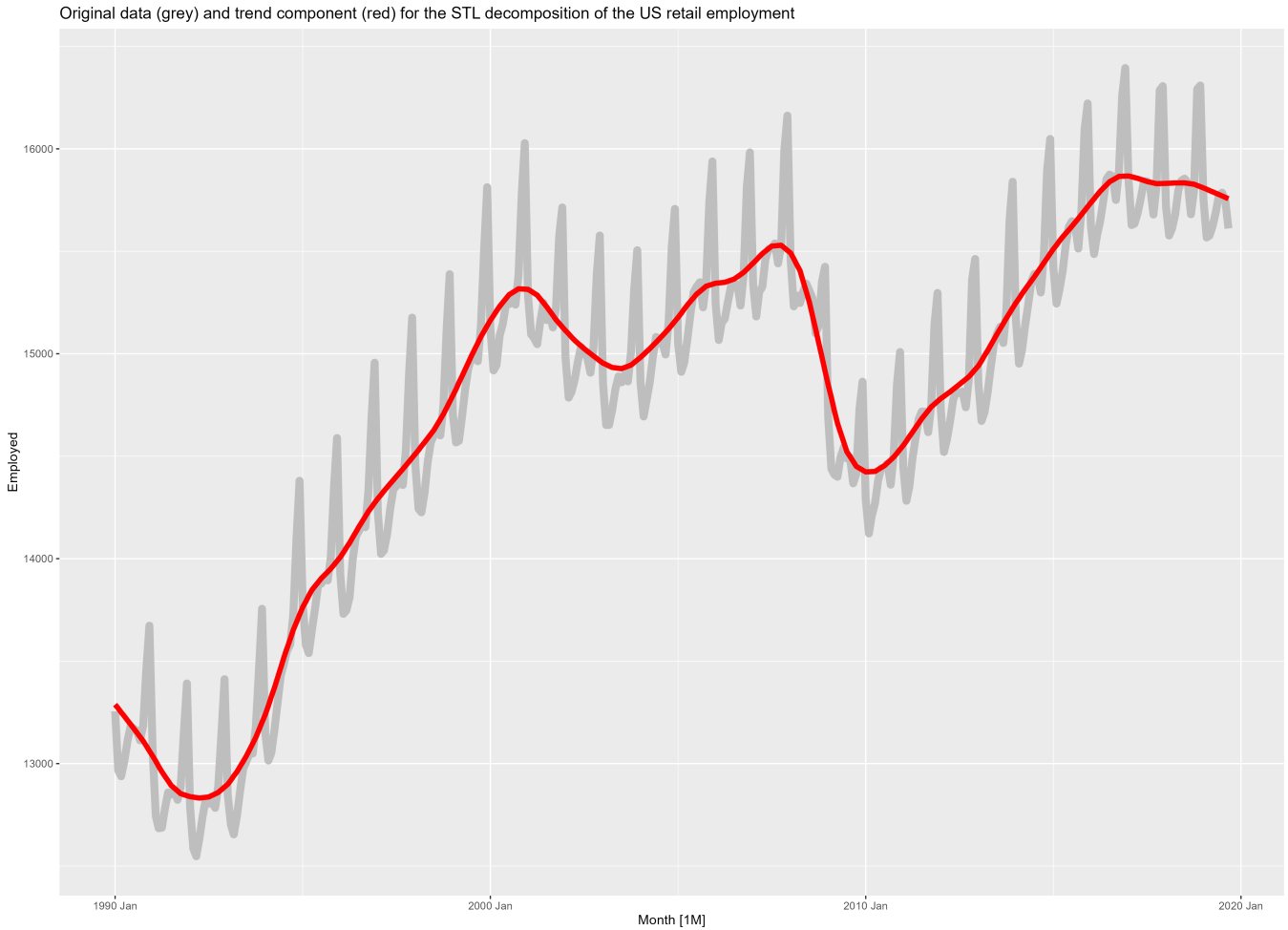
- We can use *autoplot* to plot the components of the decomposition

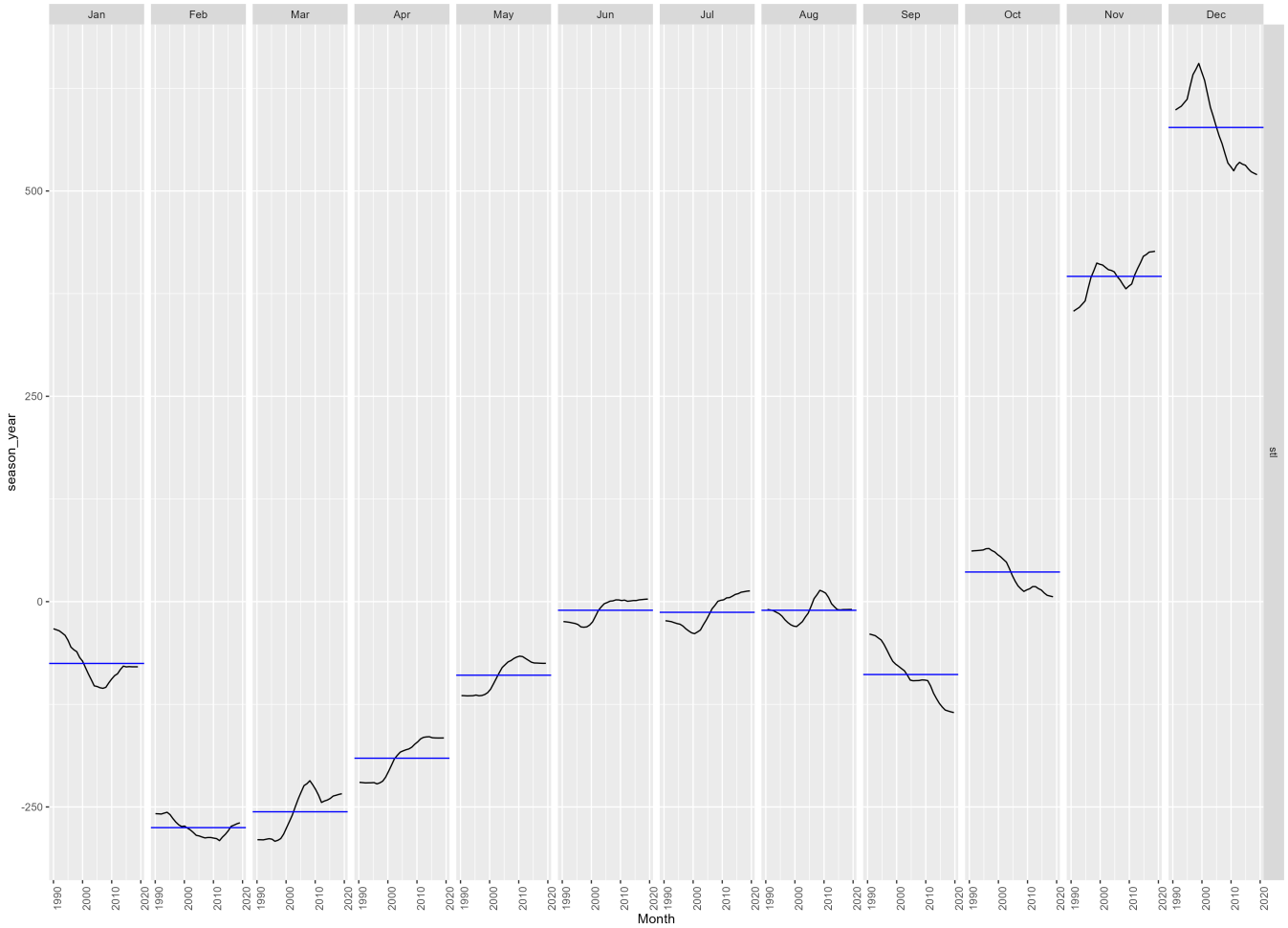


- The first subplot contains the original data.
- The second subplot contains the trend component, which is a smoother line, following the general direction of the time series.
- The third subplot is the seasonal component which looks very regular.
- The last subplot is the remainder (what remains when the subtract from the original data the trend and the season).
- All 4 panels have the same size, but the data they contain is on different scales. The remained varies between -100 and 100, while the trend varies between aprox. 12500 and 16000. This variation is denoted by the size of the grey bars on the left.

- We can create different plots using the decomposition data:
  - We can plot the original data and the trend component
  - We can create seasonal subseries plots for the seasonal component to see how the seasons change.

```
us_retail_employment |>
  autoplot(Employed, color = "gray", size = 3) +
  autolayer(components(dcmp), trend, color = "red", size = 2) +
  labs(title = "Original data (grey) and trend component (red) for the STL
      decomposition of the US retail employment")

components(dcmp) |> gg_subseries(season_year)
```

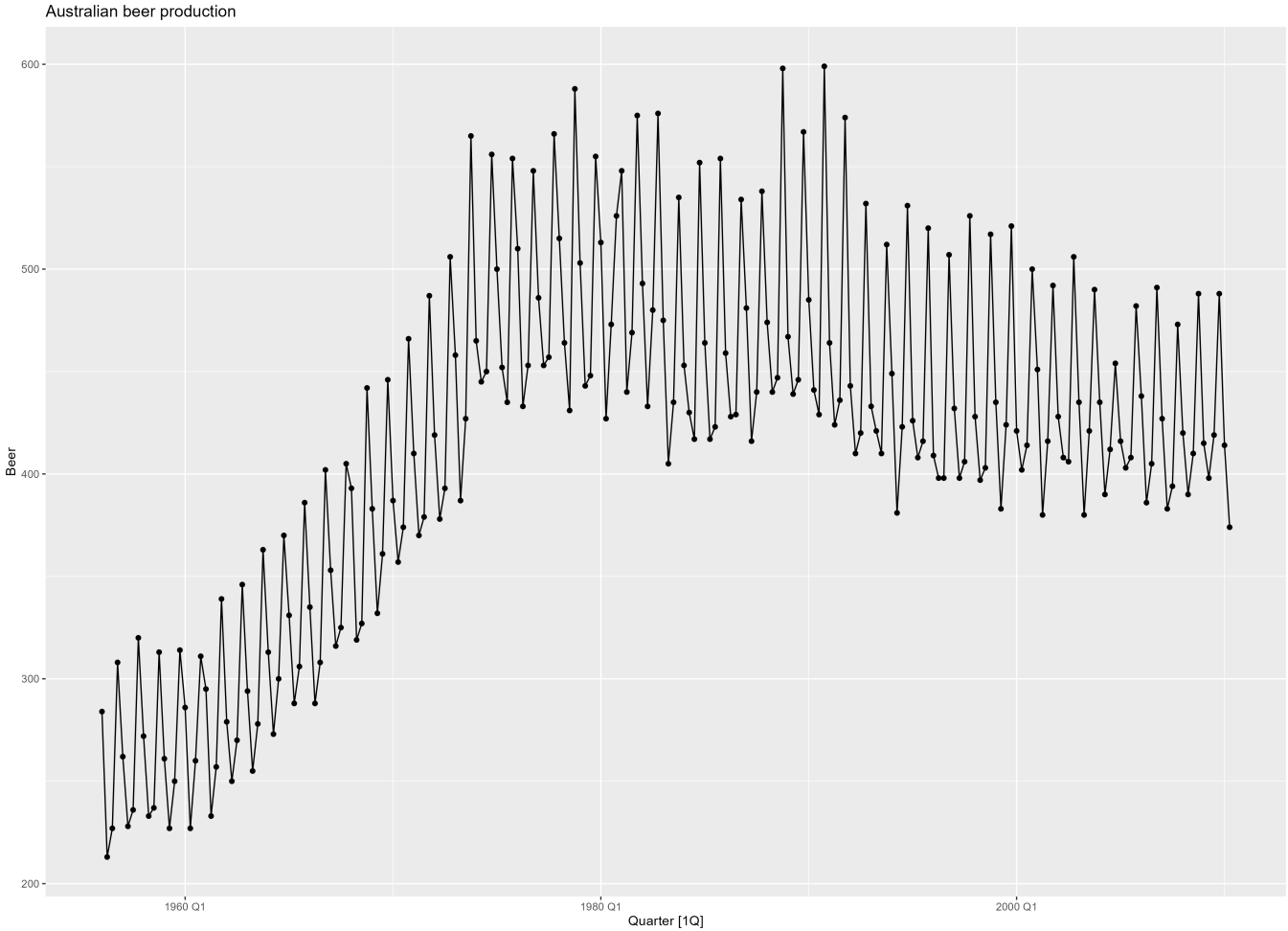

Original data (grey) and trend component (red) for the STL decomposition of the US retail employment
Employed
16000
15000
14000
13000
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month [1M]


Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
season_year
500
250
0
-250
1990
2000
2010
2020
Month
stl

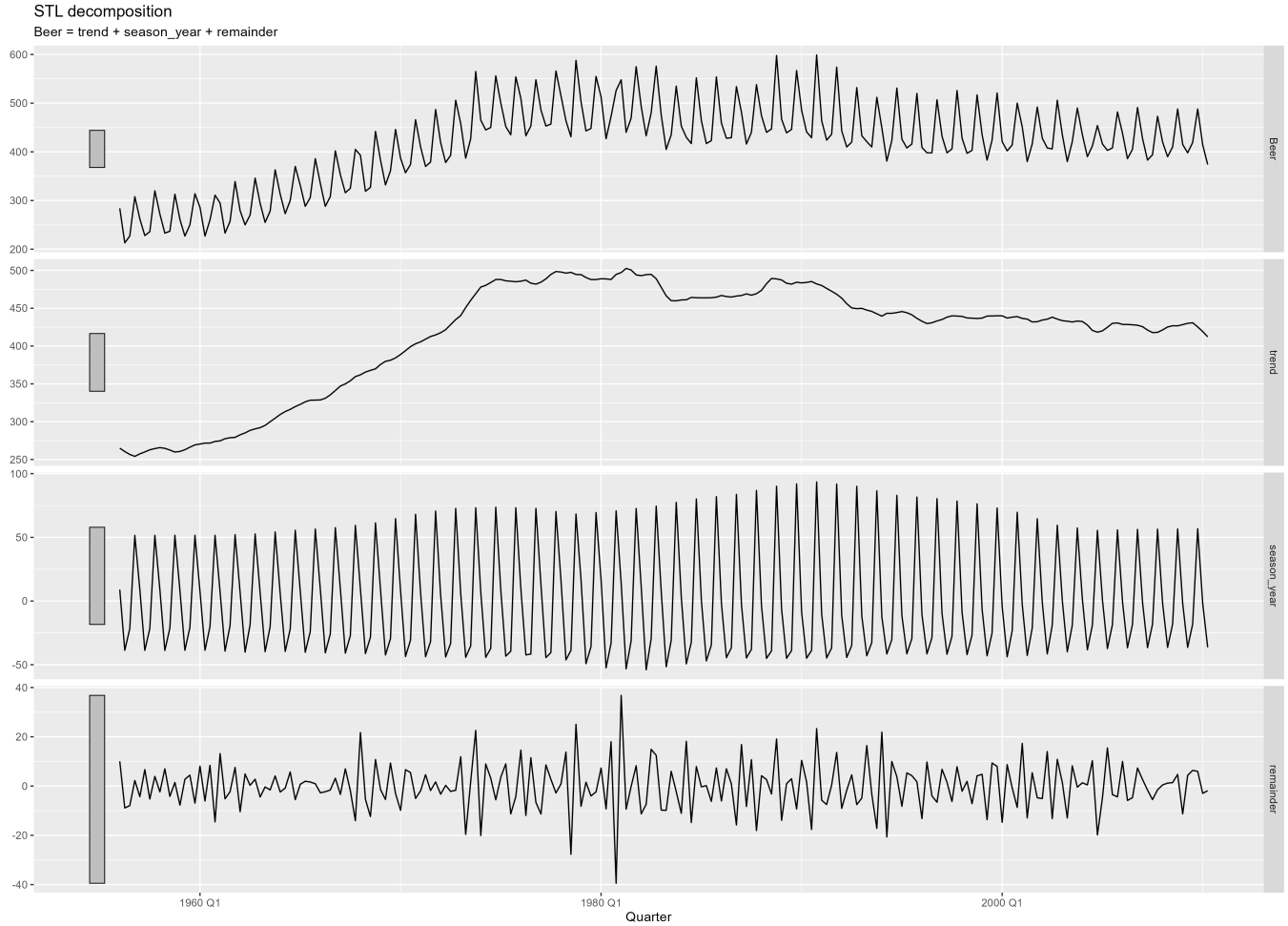
Other examples

- Quarterly Australian beer production

```
beer <- aus_production |>
  select(Quarter, Beer)

beer |> autoplot() +
  geom_point() +
  labs(title = "Australian beer production")

dcmpBeer <- beer |>
  model(stl = STL(Beer))

components(dcmpBeer)

components(dcmpBeer) |>
  autoplot()
```


Australian beer production
600
500
400
300
200
Beer
1960 Q1
1980 Q1
2000 Q1
Quarter [1Q]

```
> components(dcmpBeer)
# A dable: 218 x 7 [1Q]
# Key:     .model [1]
# :        Beer = trend + season_year + remainder
   .model Quarter  Beer trend season_year remainder season_adjust
   <chr>    <qtr> <dbl> <dbl>       <dbl>     <dbl>          <dbl>
 1 stl    1956 Q1   284  265.        8.97     10.1            275.
 2 stl    1956 Q2   213  261.      -38.7      -8.88           252.
 3 stl    1956 Q3   227  257.      -21.8      -7.97           249.
 4 stl    1956 Q4   308  254.       51.6       2.25           256.
 5 stl    1957 Q1   262  257.        8.81     -4.31           253.
 6 stl    1957 Q2   228  260.      -38.7       6.70           267.
 7 stl    1957 Q3   236  263.      -21.6      -5.28           258.
 8 stl    1957 Q4   320  264.       51.6       3.91           268.
 9 stl    1958 Q1   272  266.        8.66     -2.39           263.
10 stl    1958 Q2   233  265.      -38.8       7.01           272.
# i 208 more rows
# i Use `print(n = ...)` to see more rows
```

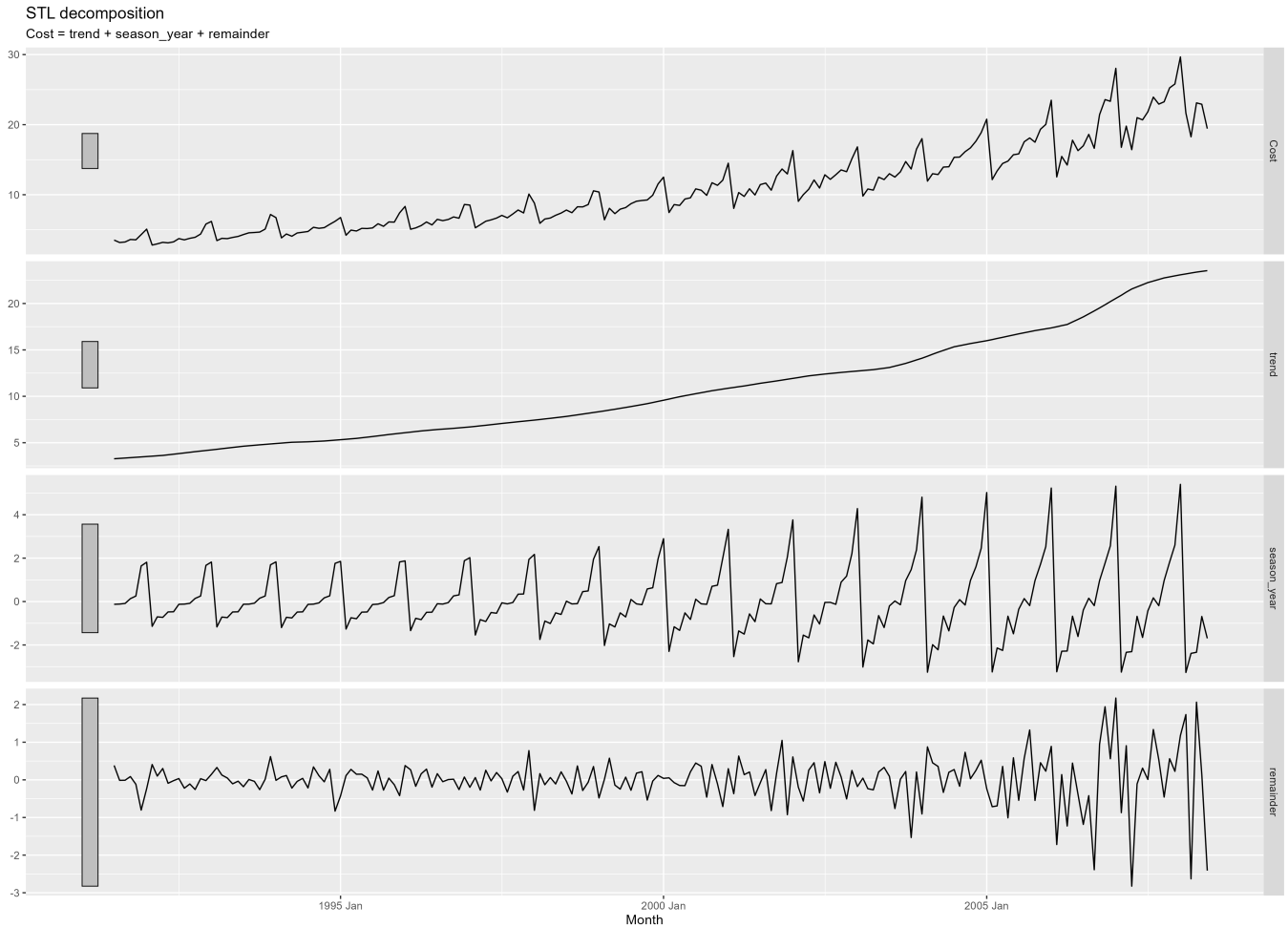

STL decomposition
Beer = trend + season_year + remainder
600
500
400
300
200
Beer
500
450
400
350
300
250
trend
100
50
0
-50
season_year
40
20
0
-20
-40
remainder
1960 Q1
1980 Q1
2000 Q1
Quarter

Other examples

- Antidiabetic drug sales

```
a10 |> autoplot()

dcmp_a10 <- a10 |>
  model(stl = STL(Cost))

components(dcmp_a10)

components(dcmp_a10) |>
  autoplot()
```

```
# A dable: 204 x 7 [1M]
# Key:     .model [1]
# :        Cost = trend + season_year + remainder
   .model    Month  Cost trend season_year remainder season_adjust
   <chr>     <mth> <dbl> <dbl>       <dbl>     <dbl>          <dbl>
 1 stl    1991 Jul  3.53  3.27      -0.127     0.383           3.65
 2 stl    1991 Aug  3.18  3.31      -0.116    -0.0129          3.30
 3 stl    1991 Sep  3.25  3.35      -0.0872   -0.0104          3.34
 4 stl    1991 Oct  3.61  3.39       0.134     0.0872          3.48
 5 stl    1991 Nov  3.57  3.43       0.254    -0.118           3.31
 6 stl    1991 Dec  4.31  3.47       1.64     -0.804           2.67
 7 stl    1992 Jan  5.09  3.51       1.81     -0.233           3.28
 8 stl    1992 Feb  2.81  3.55      -1.14      0.408           3.96
 9 stl    1992 Mar  2.99  3.59      -0.704     0.0999          3.69
10 stl    1992 Apr  3.20  3.63      -0.727     0.302           3.93
# i 194 more rows
# i Use `print(n = ...)` to see more rows
```

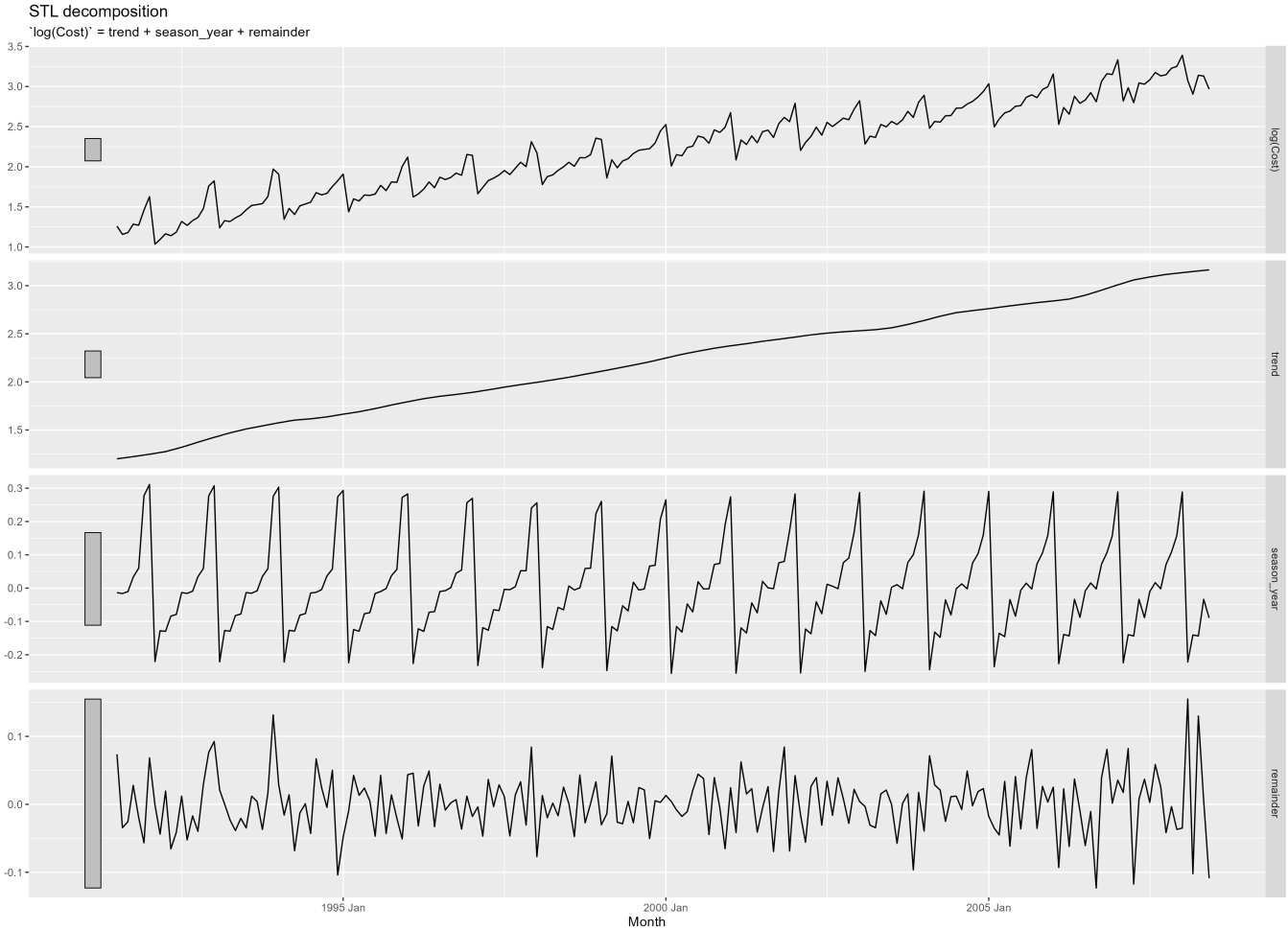


- What do you notice on this plot?

- The antidiabetic drug sales time series was one with multiplicative seasonality.
- We cannot do multiplicative decomposition with STL, and on the seasonal component of the additive decomposition we can see that something is not right.
- We can transform the data applying a log transformation and then do the decomposition.

```
dcmp_a10L <- a10 |>
    model(stl = STL(log(Cost)))

components(dcmp_a10L)

components(dcmp_a10L) |>
    autoplot()
```

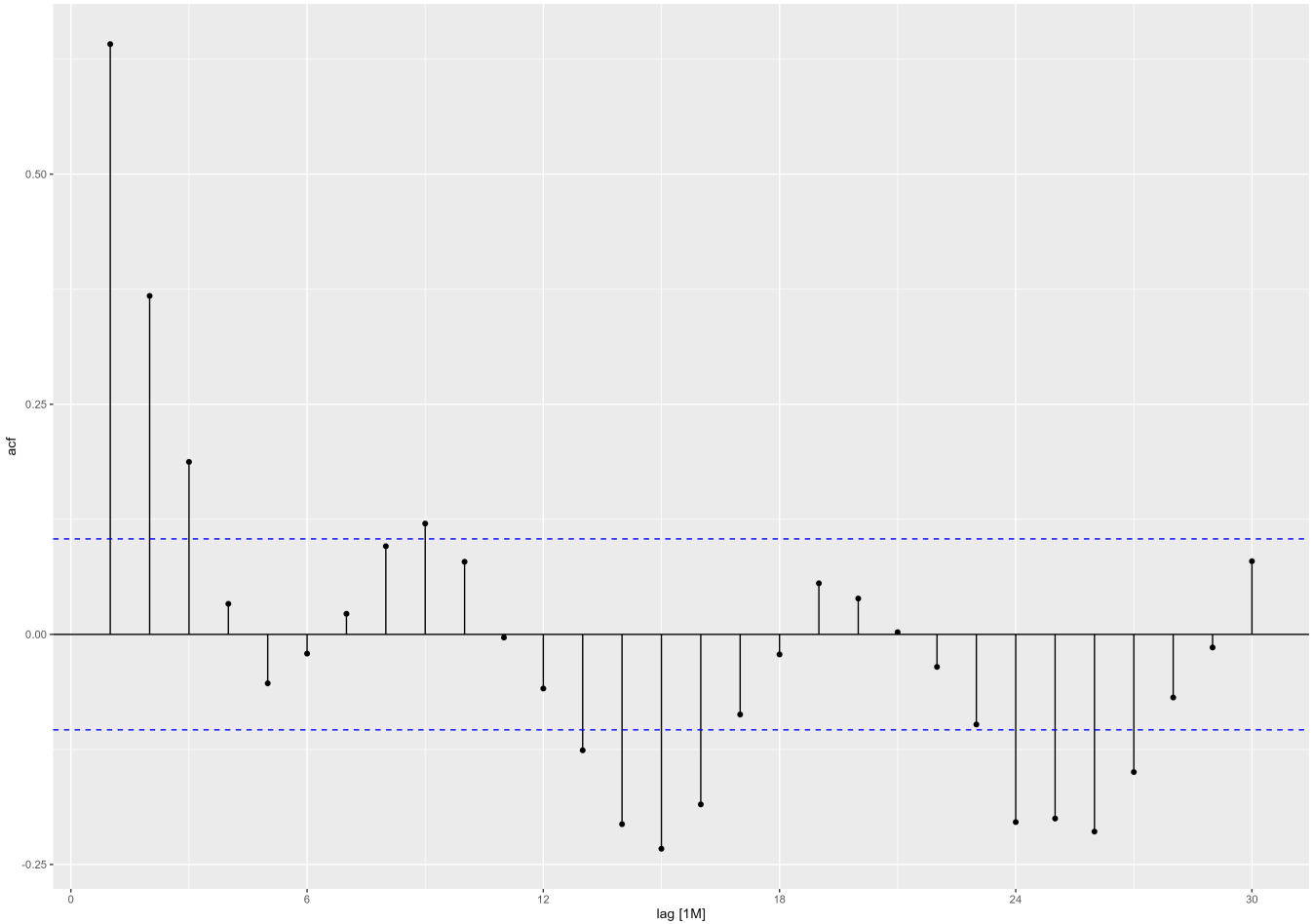
```
# A dable: 204 x 7 [1M]
# Key:     .model [1]
# :        log(Cost) = trend + season_year + remainder
   .model    Month `log(Cost)` trend season_year remainder season_adjust
   <chr>     <mth>       <dbl> <dbl>       <dbl>     <dbl>          <dbl>
 1 stl    1991 Jul        1.26  1.20     -0.0136  0.0736           1.27
 2 stl    1991 Aug        1.16  1.21     -0.0162 -0.0344           1.17
 3 stl    1991 Sep        1.18  1.22     -0.0105 -0.0255           1.19
 4 stl    1991 Oct        1.28  1.22      0.0332  0.0279           1.25
 5 stl    1991 Nov        1.27  1.23      0.0600 -0.0196           1.21
 6 stl    1991 Dec        1.46  1.24      0.278  -0.0567           1.18
 7 stl    1992 Jan        1.63  1.25      0.311   0.0684           1.32
 8 stl    1992 Feb        1.03  1.26     -0.221  -0.000911         1.26
 9 stl    1992 Mar        1.09  1.27     -0.128  -0.0439           1.22
10 stl    1992 Apr        1.16  1.27     -0.130   0.0195           1.29
# i 194 more rows
# i Use `print(n = ...)` to see more rows
```

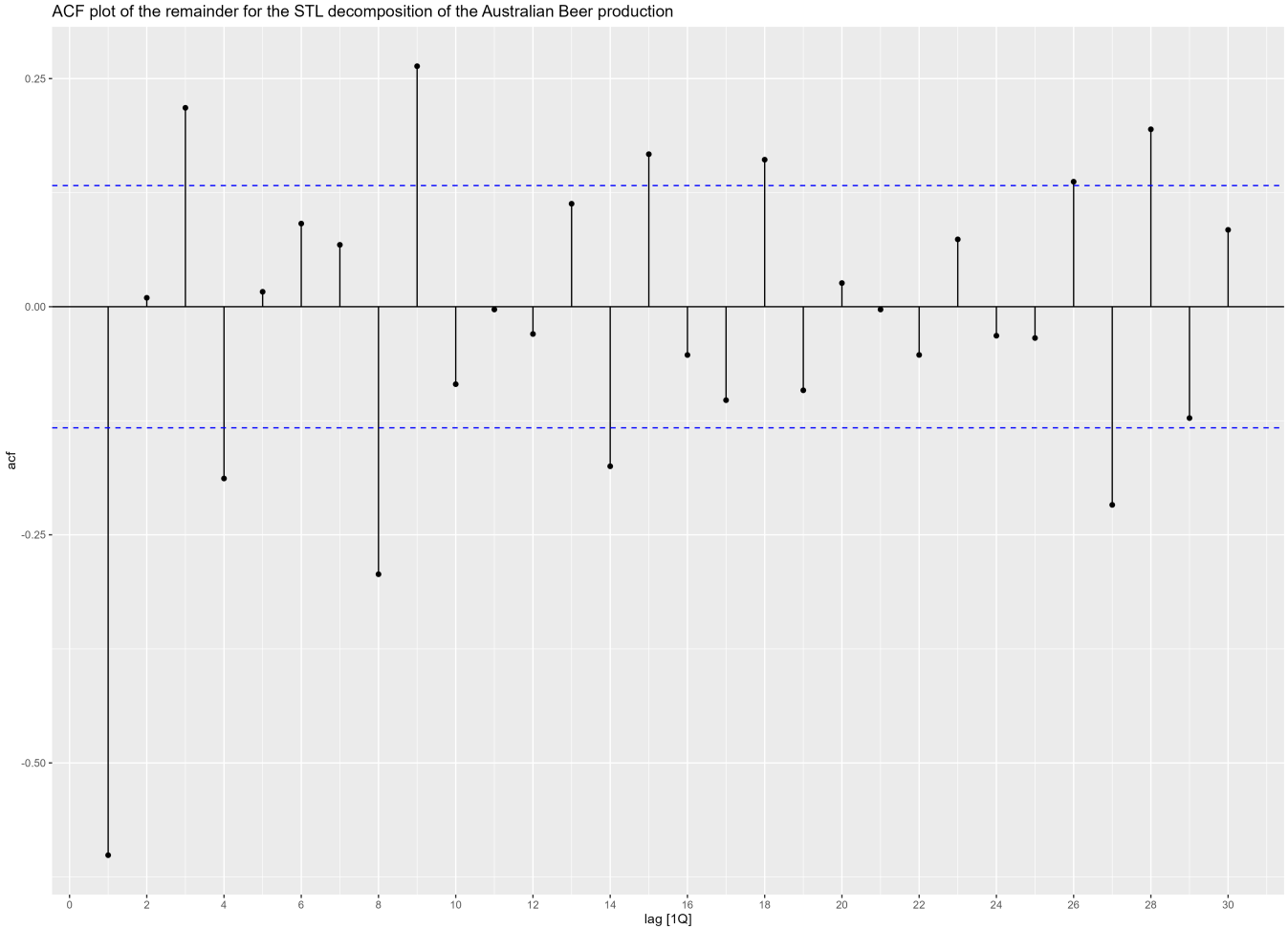

STL decomposition
`log(Cost)` = trend + season_year + remainder
log(Cost)
trend
season_year
remainder
1995 Jan
2000 Jan
2005 Jan
Month

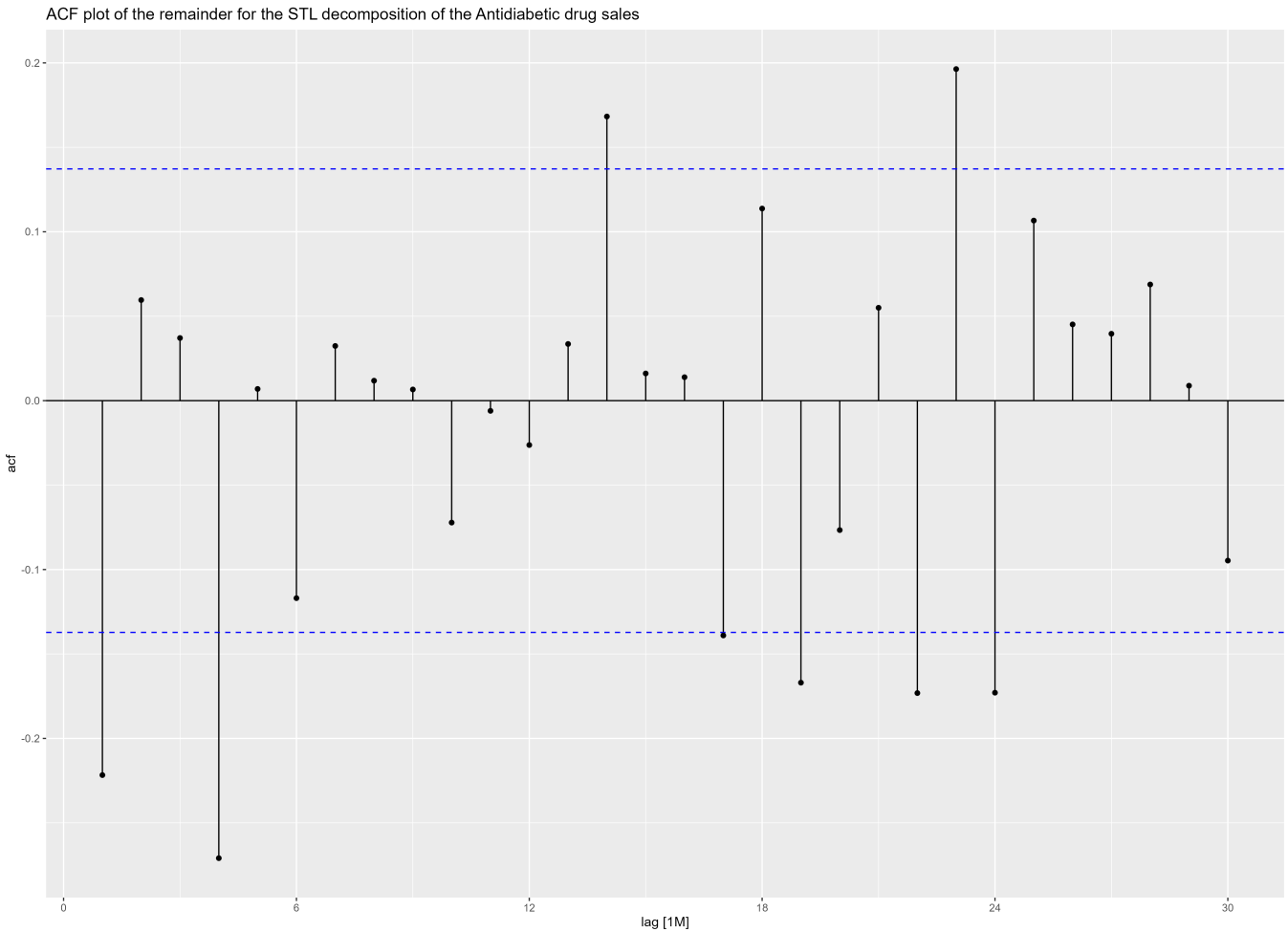
- A decomposition is said to be a good one if the remainder is white noise (meaning that all patterns in data were captured by the trend and season components).
- We can check for white noise using the ACF plot.

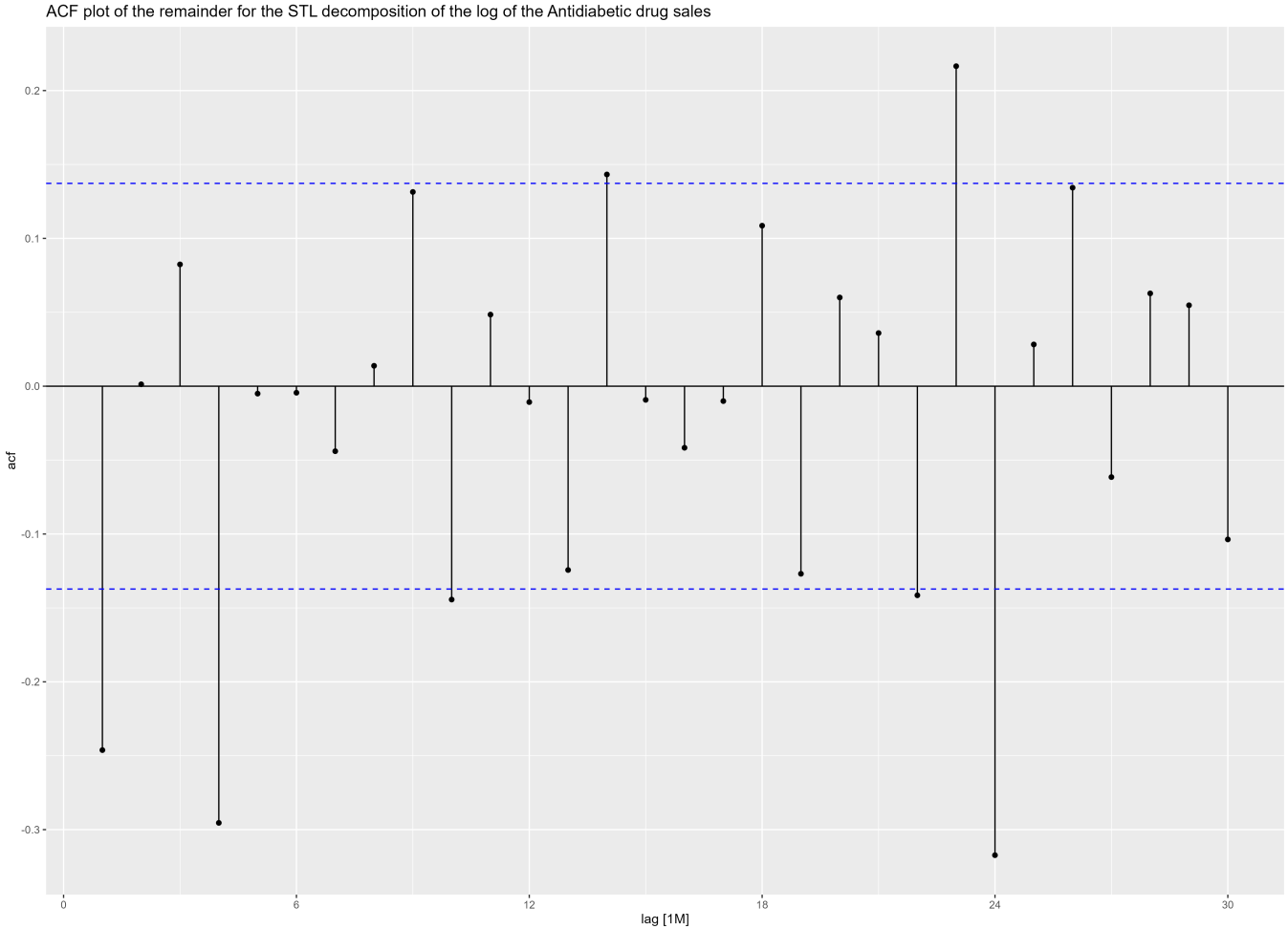
```
components(dcmp) |>
  ACF(remainder, lag_max = 30) |>
  autoplot() +
  geom_point() +
  labs(title = "ACF plot of the remainder for the STL decomposition of the US
      retail employment")
components(dcmpBeer) |>
  ACF(remainder, lag_max = 30) |>
  autoplot() +
  geom_point() +
  labs(title = "ACF plot of the remainder for the STL decomposition of the
      Australian Beer production")
components(dcmp_a10) |>
  ACF(remainder, lag_max = 30) |>
  autoplot() +
  geom_point() +
  labs(title = "ACF plot of the remainder for the STL decomposition of the
      Antidiabetic drug sales")
components(dcmp_a10L) |>
  ACF(remainder, lag_max = 30) |>
  autoplot() +
  geom_point() +
  labs(title = "ACF plot of the remainder for the STL decomposition of the log
      of the Antidiabetic drug sales")
```

ACF plot of the remainder for the STL decomposition of the US retail employment




ACF plot of the remainder for the STL decomposition of the Australian Beer production
0.25
0.00
-0.25
-0.50
acf
0
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
lag [1Q]

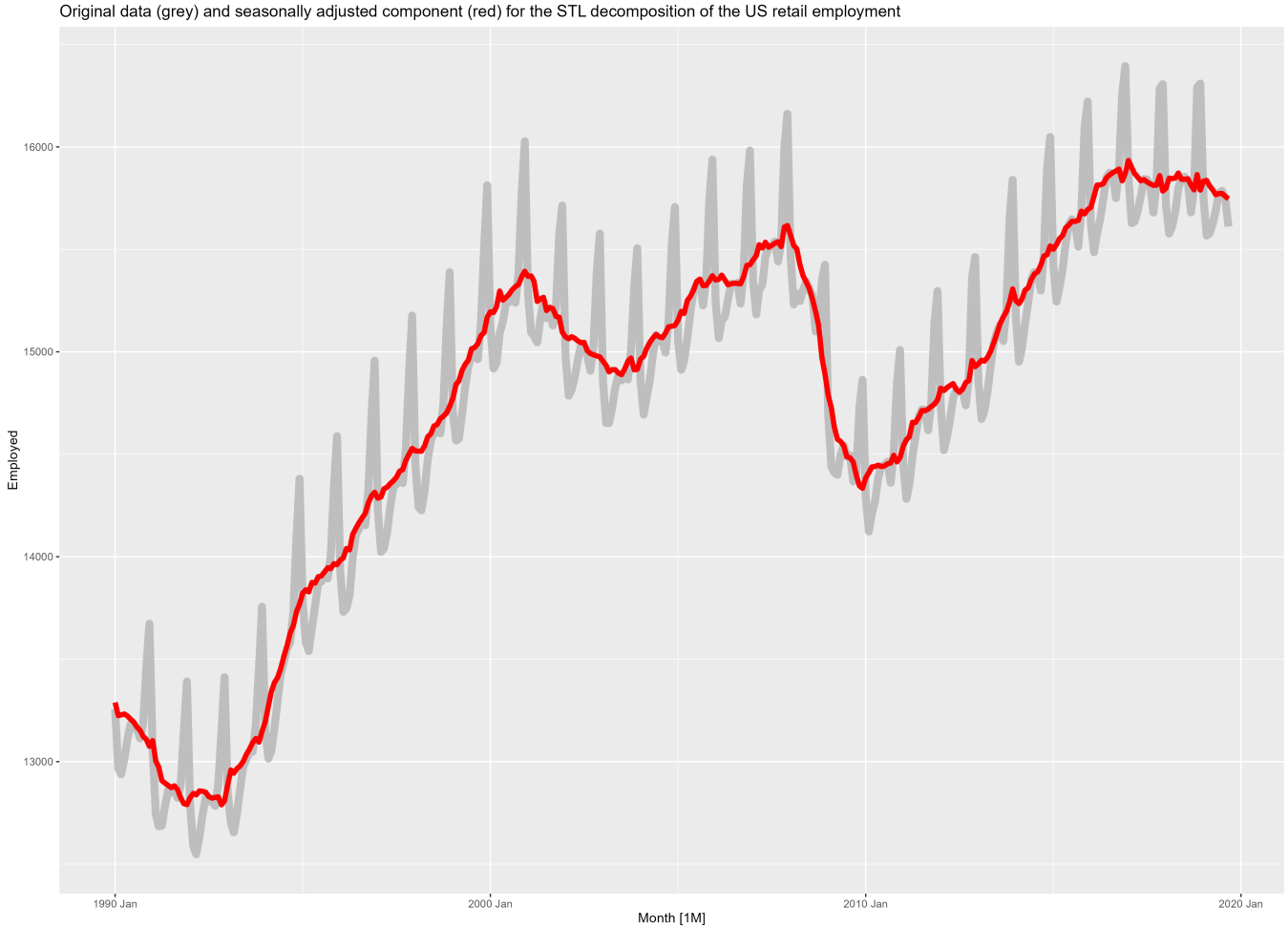

ACF plot of the remainder for the STL decomposition of the Antidiabetic drug sales
acf
0.2
0.1
0.0
-0.1
-0.2
0
6
12
18
24
30
lag [1M]


ACF plot of the remainder for the STL decomposition of the log of the Antidiabetic drug sales
acf
0.2
0.1
0.0
-0.1
-0.2
-0.3
0
6
12
18
24
30
lag [1M]

Seasonally adjusted data

- The last column in the dabble was called *season_adjust* and it contains seasonally adjusted data, i.e. data from which the seasonal component was removed.
- In our example where an additive decomposition was used, the seasonally adjusted data is just the trend + noise component. In case of a multiplicative decomposition the seasonally adjusted data is the trend * noise.
- Let us plot the original Employment data and the seasonally adjusted component together

```
us_retail_employment |>
  autoplot(Employed, color = "gray", size = 3) +
  autolayer(components(dcmp), season_adjust, color = "red", size = 2) +
  labs(title = "Original data (grey) and seasonally adjusted component (red) for
       the STL decomposition of the US retail employment")
```

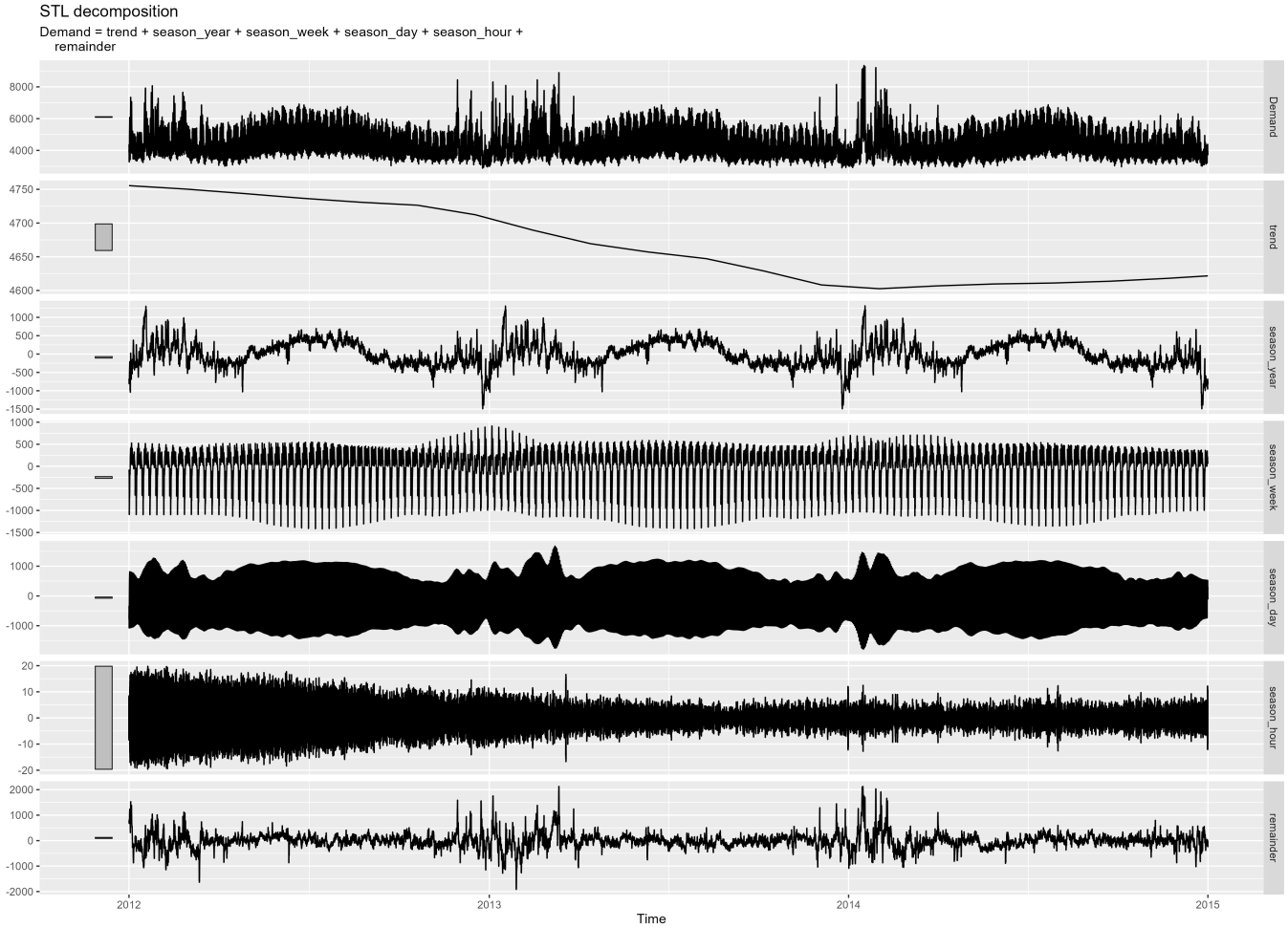

Original data (grey) and seasonally adjusted component (red) for the STL decomposition of the US retail employment
Employed
16000
15000
14000
13000
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month [1M]

- Let us see another example with STL.
- Consider the *Demand* from the *vic_elec* time series.

```
vic_elec |>
  autoplot(Demand)

dcmp_vicElec <- vic_elec |>
  model(stl = STL(Demand))

dcmp_vicElec |> components()

dcmp_vicElec |>
  components() |>
  autoplot()
```

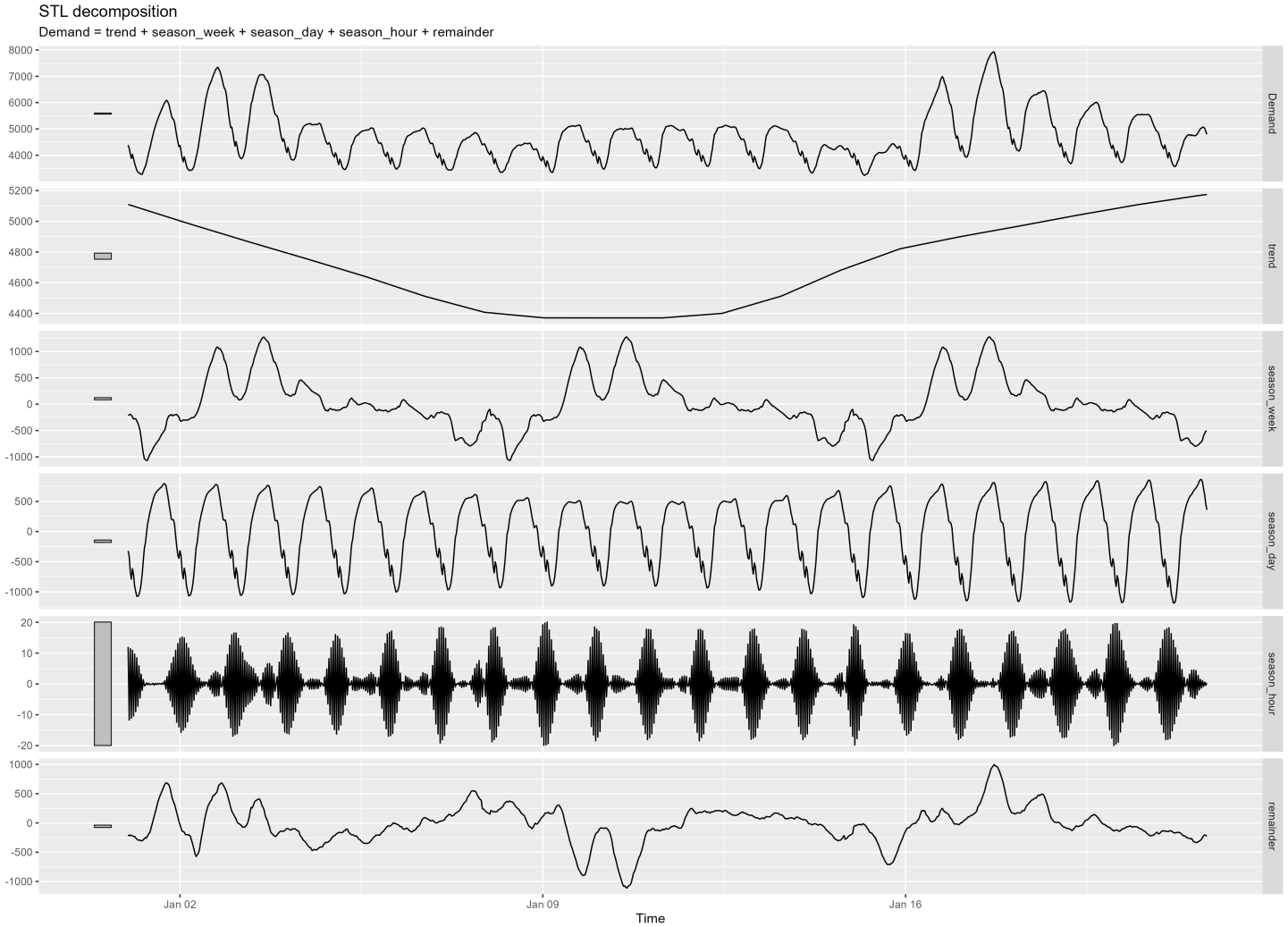
```
> dcmp_vicElec |> components()
# A dable: 52,608 x 10 [30m] <Australia/Melbourne>
# Key:     .model [1]
# :        Demand = trend + season_year + season_week + season_day + season_hour + remainder
   .model Time                Demand trend season_hour season_day season_week season_year remainder season_adjust
   <chr>  <dttm>               <dbl> <dbl>       <dbl>      <dbl>       <dbl>       <dbl>     <dbl>          <dbl>
 1 stl    2012-01-01 00:00:00  4383. 4756.        8.67      -331.       -88.2       -670.      707.          5463.
 2 stl    2012-01-01 00:30:00  4263. 4756.       -8.49      -418.       -90.2       -707.      732.          5487.
 3 stl    2012-01-01 01:00:00  4049. 4756.        8.26      -667.       -86.5       -719.      757.          5513.
 4 stl    2012-01-01 01:30:00  3878. 4756.       -8.02      -781.      -100.        -759.      771.          5526.
 5 stl    2012-01-01 02:00:00  4036. 4756.        7.79      -605.      -114.        -770.      762.          5518.
 6 stl    2012-01-01 02:30:00  3866. 4756.       -7.66      -703.      -131.        -808.      760.          5516.
 7 stl    2012-01-01 03:00:00  3694. 4756.        7.11      -898.      -131.        -794.      754.          5510.
 8 stl    2012-01-01 03:30:00  3562. 4755.       -6.64      -987.      -147.        -803.      750.          5506.
 9 stl    2012-01-01 04:00:00  3433. 4755.        6.35     -1070.      -179.        -792.      712.          5468.
10 stl    2012-01-01 04:30:00  3359. 4755.       -5.22     -1076.      -228.        -780.      693.          5448.
# i 52,598 more rows
# i Use `print(n = ...)` to see more rows
> |
```

- What do you notice in the decomposition?


STL decomposition
Demand = trend + season_year + season_week + season_day + season_hour +
remainder
Demand
trend
season_year
season_week
season_day
season_hour
remainder
2012
2013
2014
2015
Time

- We can see that STL identified several seasons in the data: hourly, daily, weekly and yearly.

- Since we have so many observations, the seasons are not clearly visible. Let's do the same only for part of the data (first 1000 observations, approximately 3 weeks).

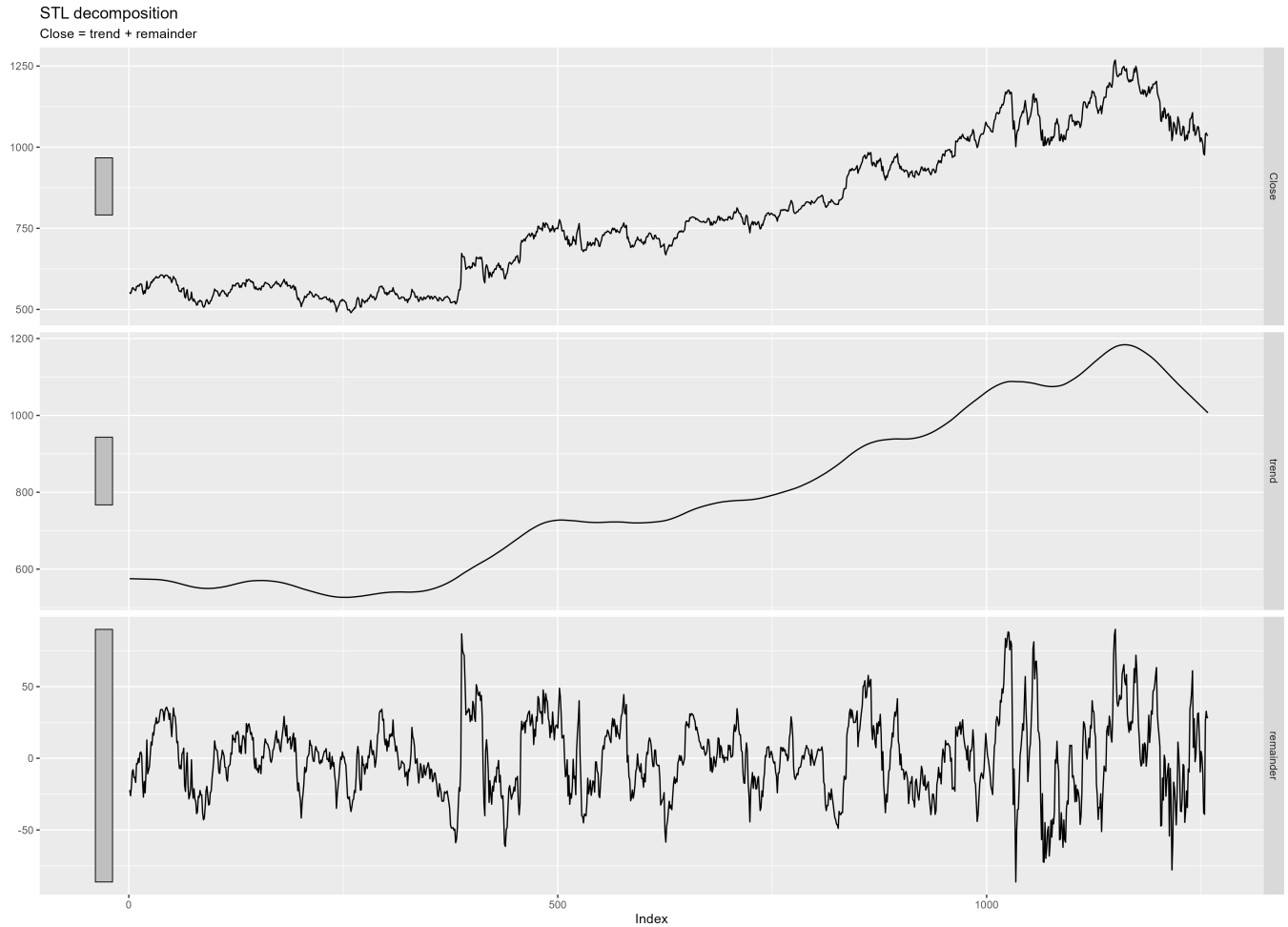

STL decomposition
Demand = trend + season_week + season_day + season_hour + remainder
Demand
trend
season_week
season_day
season_hour
remainder
Jan 02
Jan 09
Jan 16
Time

- What happens if we decompose the Google stock prices time series?

```
google <- gafa_stock |>
  filter(Symbol == "GOOG")
google

google |> autoplot(Close) +
  labs(title = "Google stock closing price")

dcmp_google <- google |>
  model(stl = STL(Close))

google <- google |>
  mutate(Index = seq(from = 1, to = 1258, by = 1))

google <-tsibble(Close = google$Close, Index = google$Index, index = Index)

google

dcmp_google <- google |>
  model(stl = STL(Close))

dcmp_google |> components()
dcmp_google |>
  components() |>
  autoplot()
```

- Since the Google data is not a regular one (it considers trading days, not days or weeks), we will have an error.
- To make the time series regular, we will reindex the observations.

```
> dcmp_google |> components()
# A dable: 1,258 x 6 [1]
# Key:     .model [1]
# :        Close = trend + remainder
   .model Index Close trend remainder season_adjust
   <chr>  <dbl> <dbl> <dbl>     <dbl>          <dbl>
 1 stl        1  553.  575.    -22.2            553.
 2 stl        2  549.  575.    -26.2            549.
 3 stl        3  555.  575.    -20.0            555.
 4 stl        4  566.  575.     -9.22           566.
 5 stl        5  567.  575.     -7.98           567.
 6 stl        6  561.  575.    -13.4            561.
 7 stl        7  561.  575.    -13.3            561.
 8 stl        8  558.  575.    -16.9            558.
 9 stl        9  571.  575.     -3.67           571.
10 stl       10  571.  575.     -3.99           571.
# i 1,248 more rows
# i Use `print(n = ...)` to see more rows
```

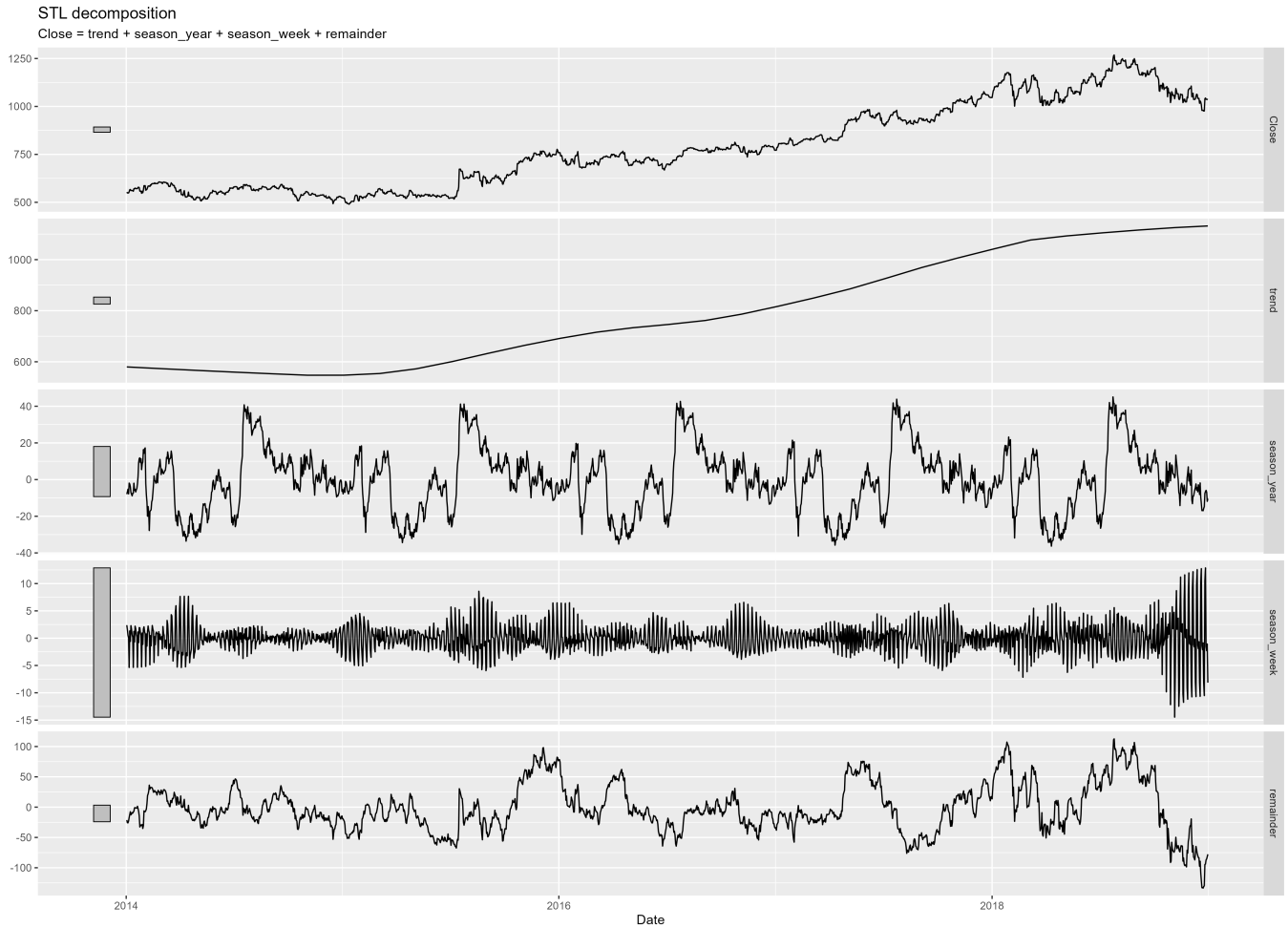


- What can you notice in the decomposition?

- We can see that there is no seasonal component.
- This is due to the fact that the time series index is just a number, for which we cannot have seasonality.
- A possible alternative solution would be to keep as index the actual date, but fill in the missing days with NA.
- However, STL decomposition gives an error if there are missing values in the time series.
- A possible solution is to fill in the missing values by simply copying the previous values.

```r
googleNoMissingFilled <- gafa_stock |>
  filter(Symbol == "GOOG")

googleNoMissingFilled  |>
  complete(Date = seq(as.Date("2014-01-02"), as.Date("2018-12-31"), by = "day"))
      |>
  as_tsibble(index = Date) |>
  fill(Close, .direction = "down") -> googleNoMissingFilled

googleNoMissingFilled

dcmp_google2 <- googleNoMissingFilled |>
  model(stl = STL(Close))

dcmp_google2 |> components()
dcmp_google2 |>
  components() |>
  autoplot()
```

```
> dcmp_google2 |> components()
# A dable: 1,825 x 8 [1D]
# Key:     .model [1]
# :        Close = trend + season_year + season_week + remainder
   .model Date       Close trend season_week season_year remainder season_adjust
   <chr>  <date>     <dbl> <dbl>       <dbl>       <dbl>     <dbl>          <dbl>
 1 stl    2014-01-02  553.  580.       2.39        -7.95    -21.3            559.
 2 stl    2014-01-03  549.  580.       1.54        -6.63    -25.7            554.
 3 stl    2014-01-04  549.  580.       0.533       -5.26    -25.9            554.
 4 stl    2014-01-05  549.  579.       0.0752      -7.79    -22.8            557.
 5 stl    2014-01-06  555.  579.      -5.37        -1.91    -17.0            562.
 6 stl    2014-01-07  566.  579.       1.33        -2.48    -12.3            567.
 7 stl    2014-01-08  567.  579.      -0.590       -4.04     -7.58           572.
 8 stl    2014-01-09  561.  579.       2.30        -7.12    -12.7            566.
 9 stl    2014-01-10  561.  579.       1.64        -7.97    -11.1            568.
10 stl    2014-01-11  561.  579.       0.621       -8.89     -9.07           570.
# i 1,815 more rows
# i Use `print(n = ...)` to see more rows
```



- What can you notice in the decomposition?

- We know that the stock price data does not normally have a seasonal component.
- The reason why the STL decomposition extracted two seasonal components is simply the fact that we have *daily* data. When you have daily data, you could have a weekly seasonality and a yearly seasonality.
- STL extracts both, but from the scales of the corresponding plots we can see that they are very small compared to the trend (and even to the remainder).

History of time series decomposition models

- The classical decomposition was developed in 1920.
- In 1957 the Census II method was introduced, which was the basis for the X-11 method and its variants (X-12-ARIMA and X-13-ARIMA)
- STL method was introduced in 1983
- SEATS and TRAMO were introduced in the 1990s.

Methods used by official statistics agencies

- Official statistics agencies have developed their own decomposition methods which are used for seasonal adjustments. Most of them are based on the so-called *X-11* or *SEATS* methods (or a combination of them), which are methods suitable for quarterly and monthly data only.
- In the *seasonal* package a series of such methods are implemented.

Moving averages

- Moving averages are needed in order to estimate the trend-cycle component in the classical decomposition.
- A moving average of order m, denoted by m-**MA** can be computed as:

$$\hat{T}_t = \frac{1}{m} \sum_{j=-k}^{k} y_{t+j}$$

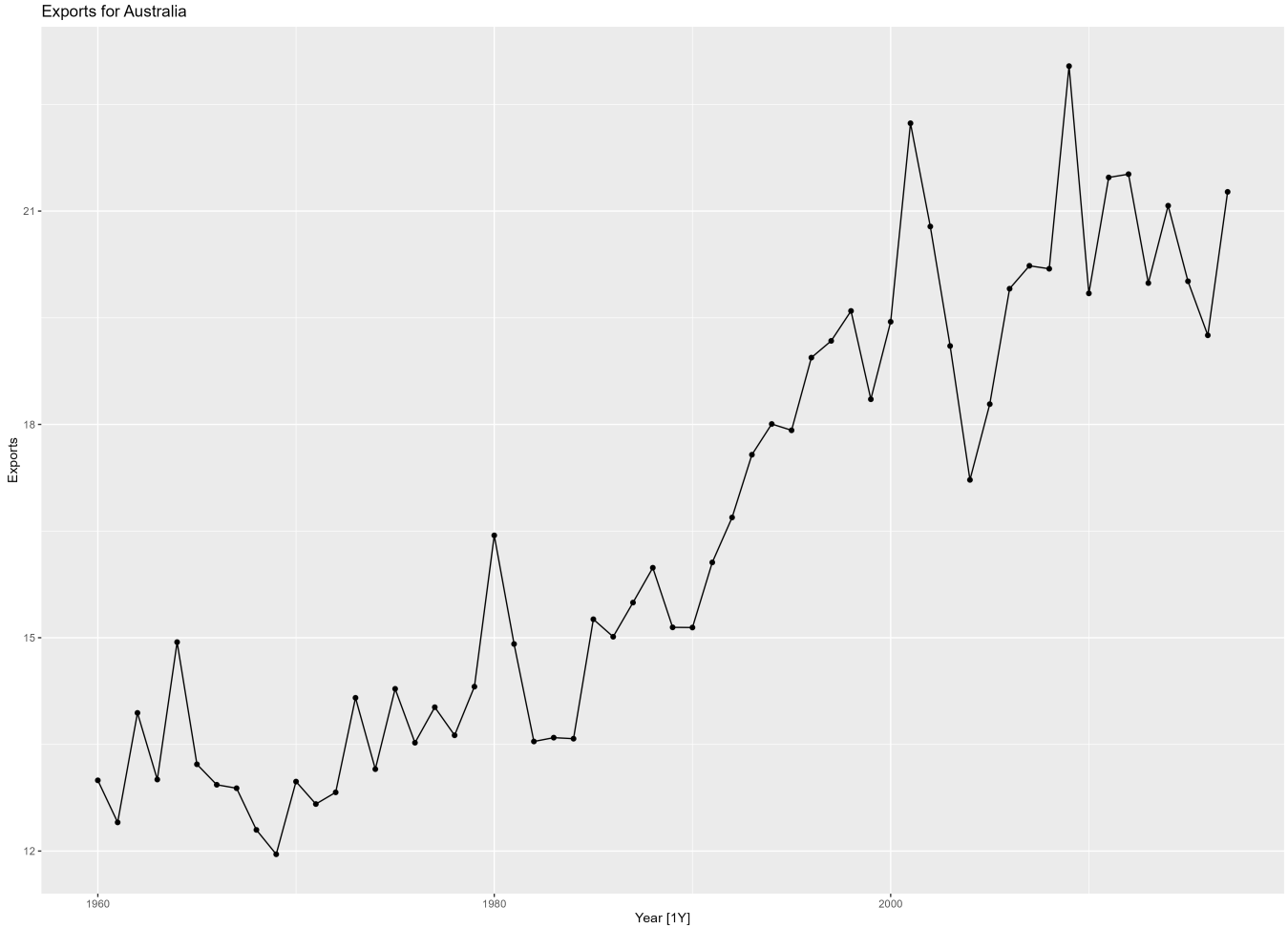
- where $m = 2k + 1$.
- In short, for estimating the moving average at a given time t, we compute the average of that value and the values for the previous and the next k observations. Assuming that observations close in time are close in value as well, the moving average will eliminate some of the randomness (will generate a smoother series).

- The moving average can be computed in R with the *slide_dbl()* function, from the *slider* package, where you need to specify the data to work on, what function to apply (in our case it is the *mean* function), how many observations to consider before and after the center one.

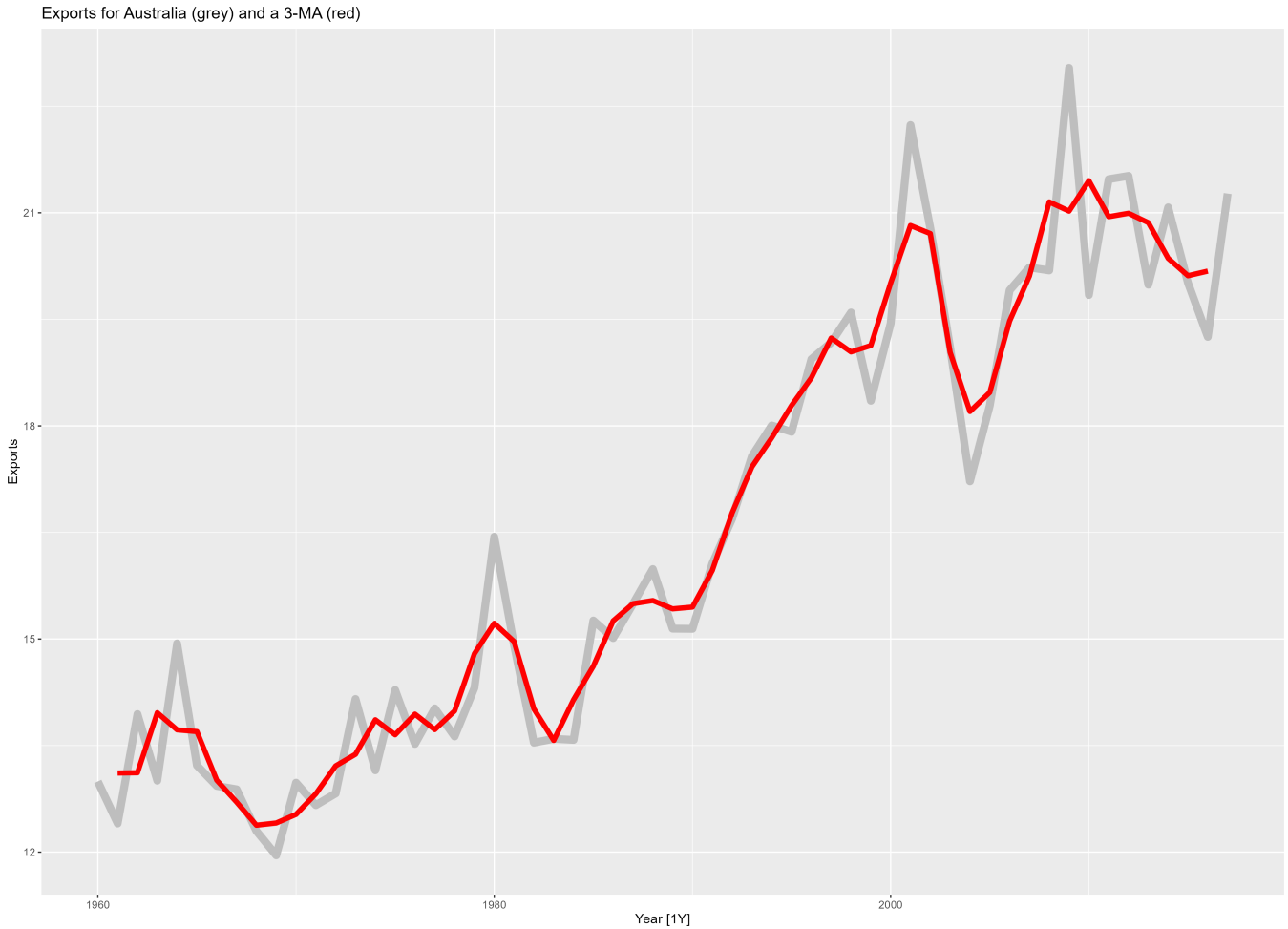
- Let us look at the yearly exports for Australia.

```
global_economy |> filter(Country == "Australia") |> select(Exports) -> aus_
    exports
aus_exports |> autoplot() + geom_point()

aus_exports_withMA <- aus_exports |>
  mutate(MA3 = slider::slide_dbl(Exports, mean, .before=1, .after = 1, .complete
      = TRUE)) |>
  mutate(MA5 = slider::slide_dbl(Exports, mean, .before=2, .after = 2, .complete
      = TRUE)) |>
  mutate(MA7 = slider::slide_dbl(Exports, mean, .before=3, .after = 3, .complete
      = TRUE))
```

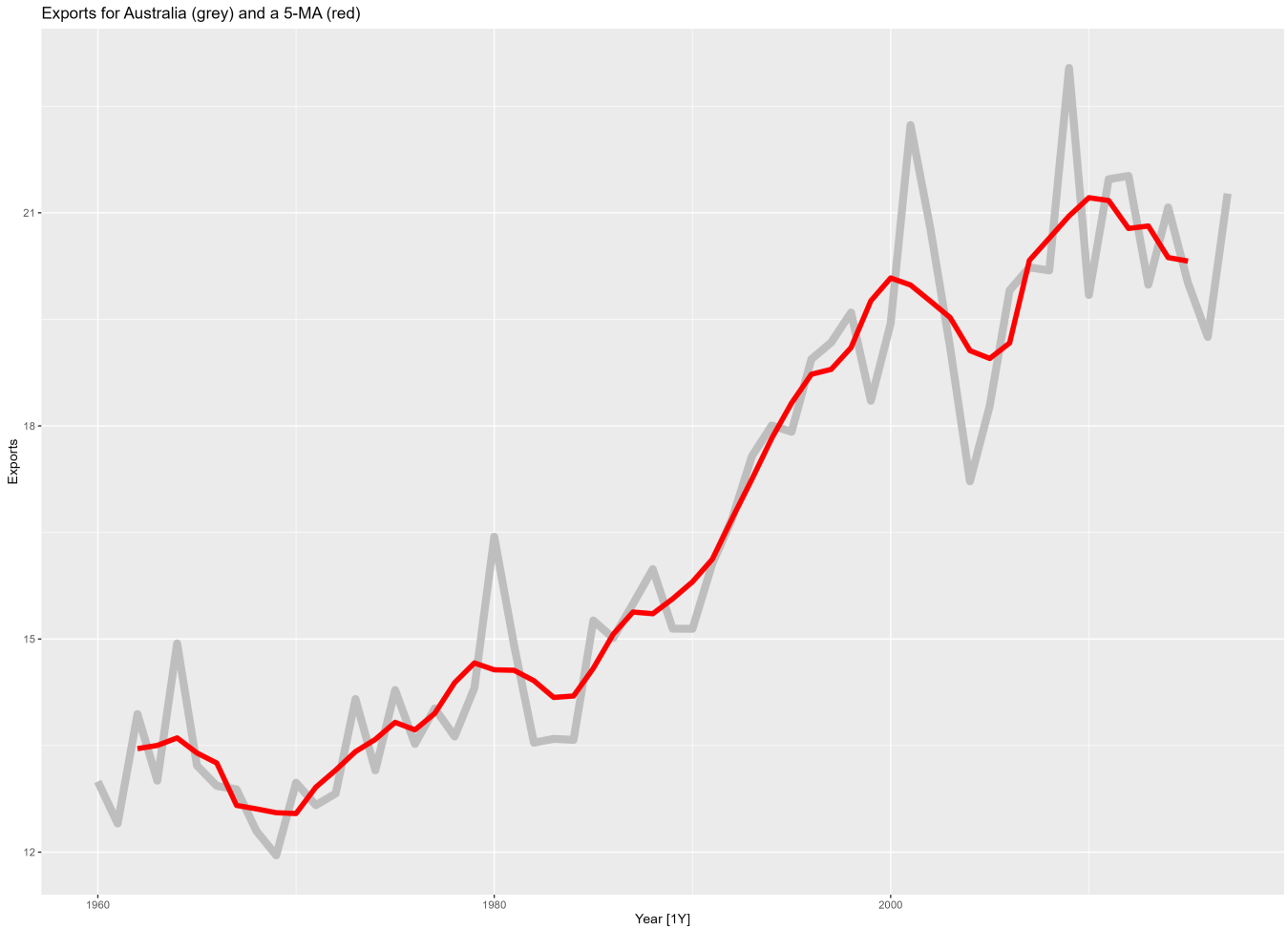
```
aus_exports_withMA |>
  autoplot(Exports, color = "grey", size = 3) +
  autolayer(aus_exports_withMA, MA3, color = "red", size = 2) +
  labs(title = "Exports for Australia (grey) and a 3–MA (red)")

aus_exports_withMA |>
  autoplot(Exports, color = "grey", size = 3) +
  autolayer(aus_exports_withMA, MA5, color = "red", size = 2) +
  labs(title = "Exports for Australia (grey) and a 5–MA (red)")

aus_exports_withMA |>
  autoplot(Exports, color = "grey", size = 3) +
  autolayer(aus_exports_withMA, MA7, color = "red", size = 2) +
  labs(title = "Exports for Australia (grey) and a 7–MA (red)")
```


Exports for Australia
Exports
21
18
15
12
1960
1980
2000
Year [1Y]

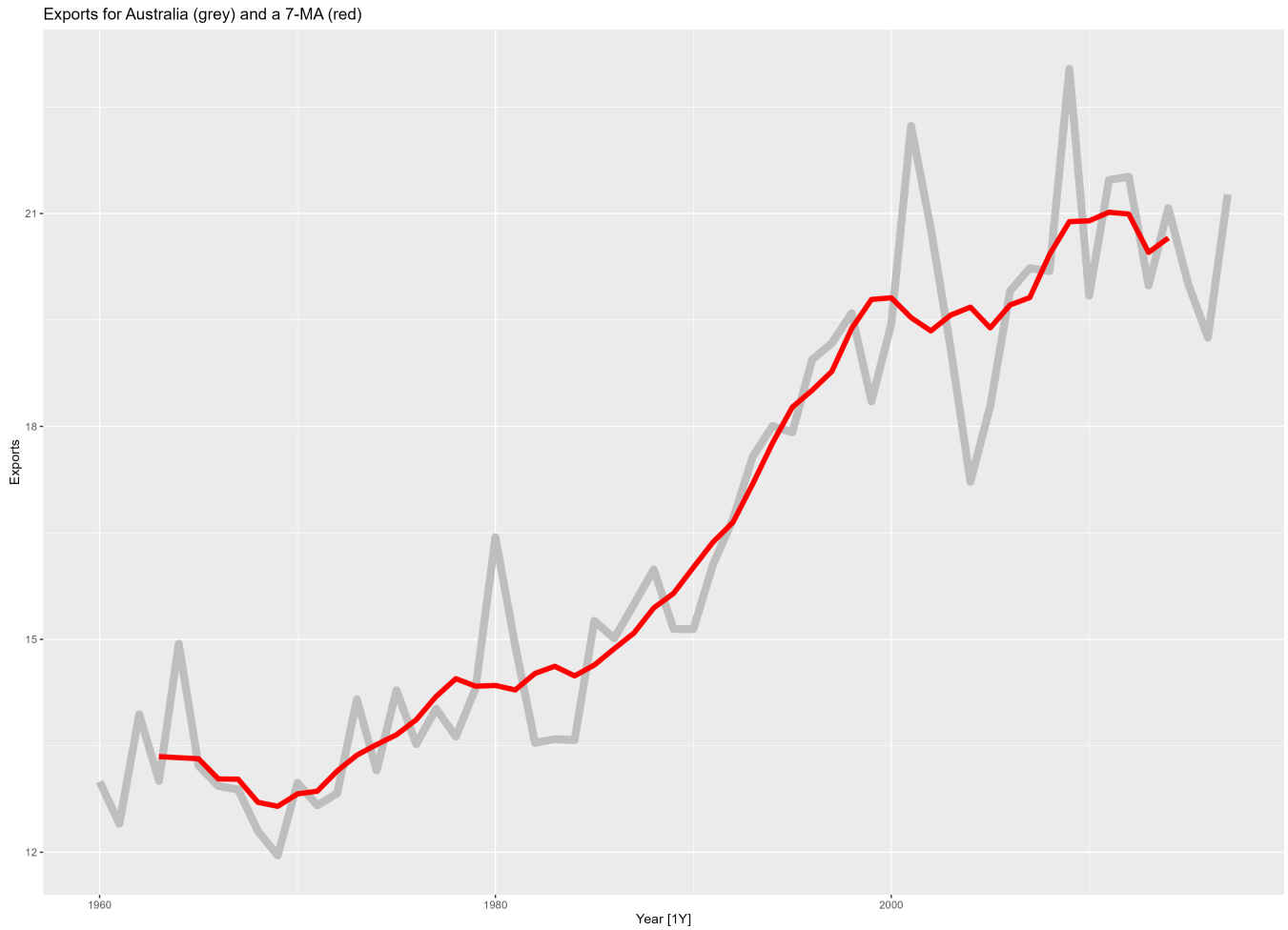
- 3-MA and original data



- 5-MA and original data



- 7-MA and original data



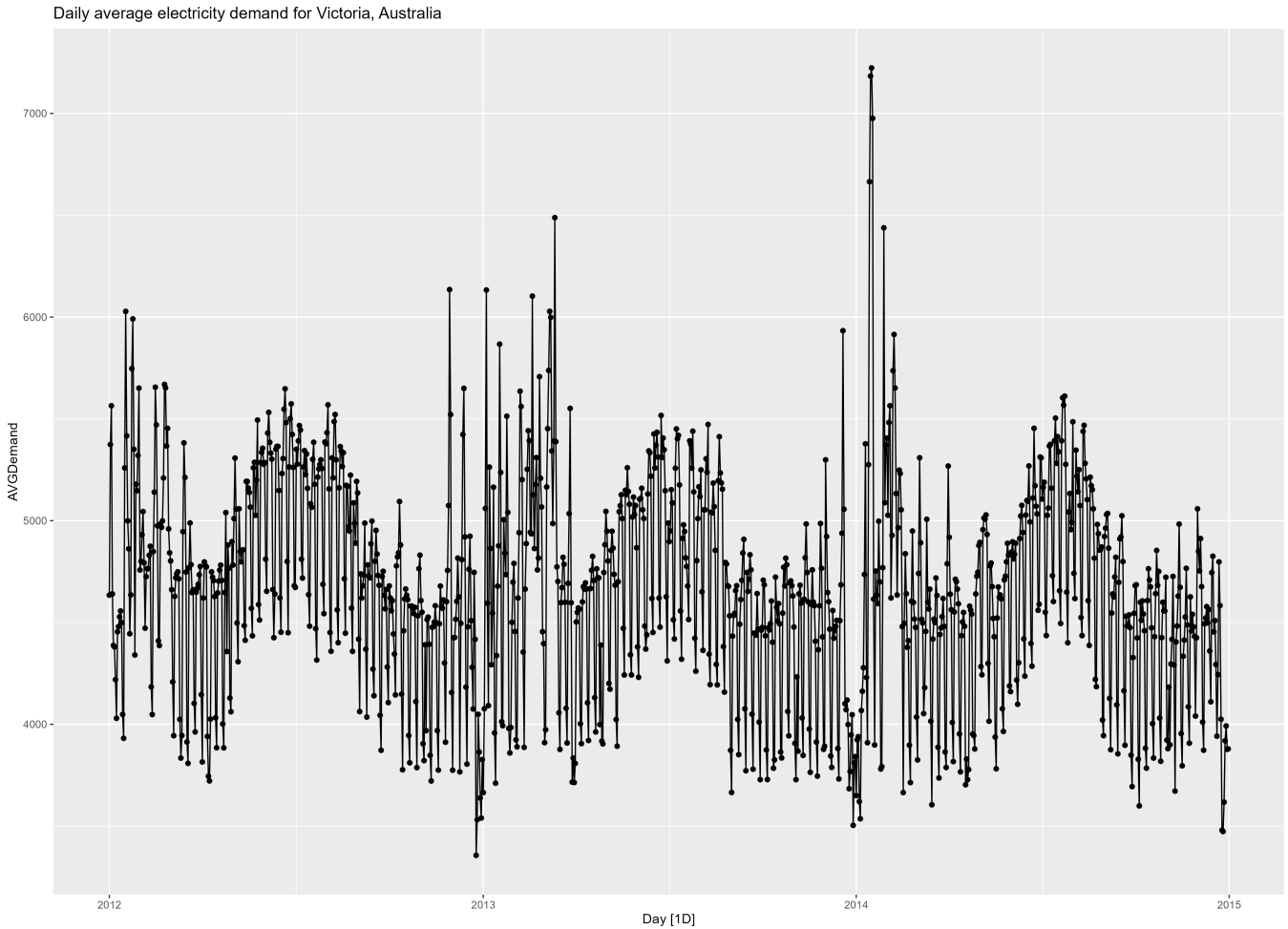
- What do you notice on this plots? How does the moving average line change, when we increase the value of m?

- Observe how the red line is shorter on the ends than our series. This is due to the fact that we did not calculate the moving average for incomplete periods. For a *5-MA* you need 2 observations before and 2 after your observation, so for the first and last two observations we cannot compute the average. This is set in *slide_dbl* with the *.complete* parameter, which flags if we want to compute the average for complete windows of observations only.
- The moving average is a smoother line than the original time series. How smooth the line is, depends on the value of m, i.e. how many observations we consider before and after. The larger the number of observations, the smoother the line.
- Simple moving averages are in general of an odd-order, so that they are symmetric.

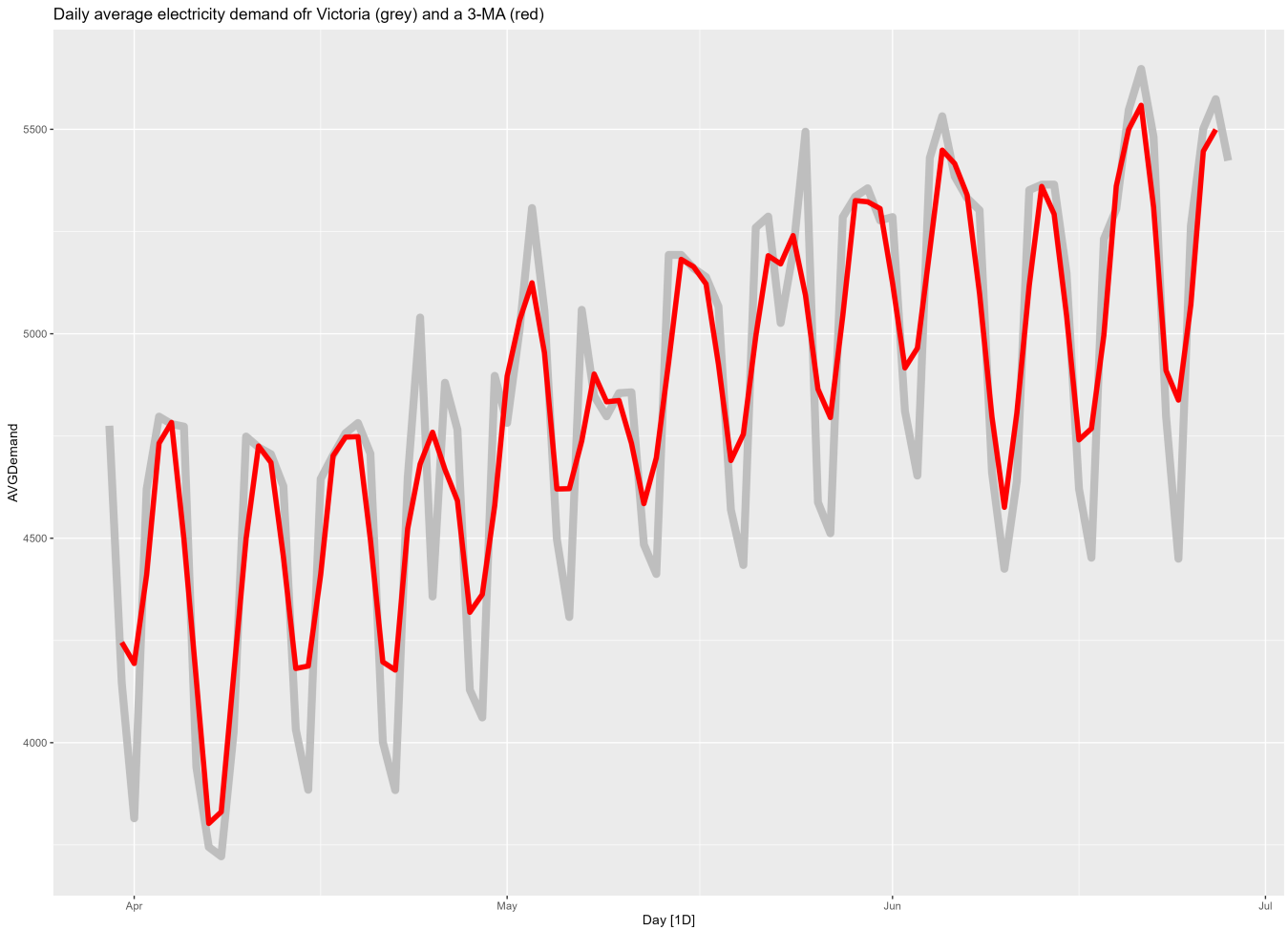
- Let us take another example, the *vic_elec* time series, but let us aggregate its data to average daily data (instead of 30 minutes frequency).
- Also, for better visibility of the plots, we will only consider approximately 3 months data, for which we will look at 3-MA, 5-MA, 7-MA, 9-MA, 11-MA.

```
vic_elec_daily <- vic_elec |>
  mutate(Day = as_date(Time)) |>
  index_by(Day) |>
  summarize(AVGDemand = mean(Demand))
vic_elec_daily |>
  autoplot() +
  geom_point() +
  labs(title = "Daily average electricity demand for Victoria, Australia")
vic_elec_daily <- vic_elec_daily |> slice(90:180)

vic_elec_daily |>
  mutate(MA3 = slider::slide_dbl(AVGDemand, mean, .before=1, .after = 1, .
      complete = TRUE)) |>
  mutate(MA5 = slider::slide_dbl(AVGDemand, mean, .before=2, .after = 2, .
      complete = TRUE)) |>
  mutate(MA7 = slider::slide_dbl(AVGDemand, mean, .before=3, .after = 3, .
      complete = TRUE)) |>
  mutate(MA9 = slider::slide_dbl(AVGDemand, mean, .before=4, .after = 4, .
      complete = TRUE)) |>
  mutate(MA11 = slider::slide_dbl(AVGDemand, mean, .before=5, .after = 5, .
      complete = TRUE)) ->vic_elec_daily_withMA
```

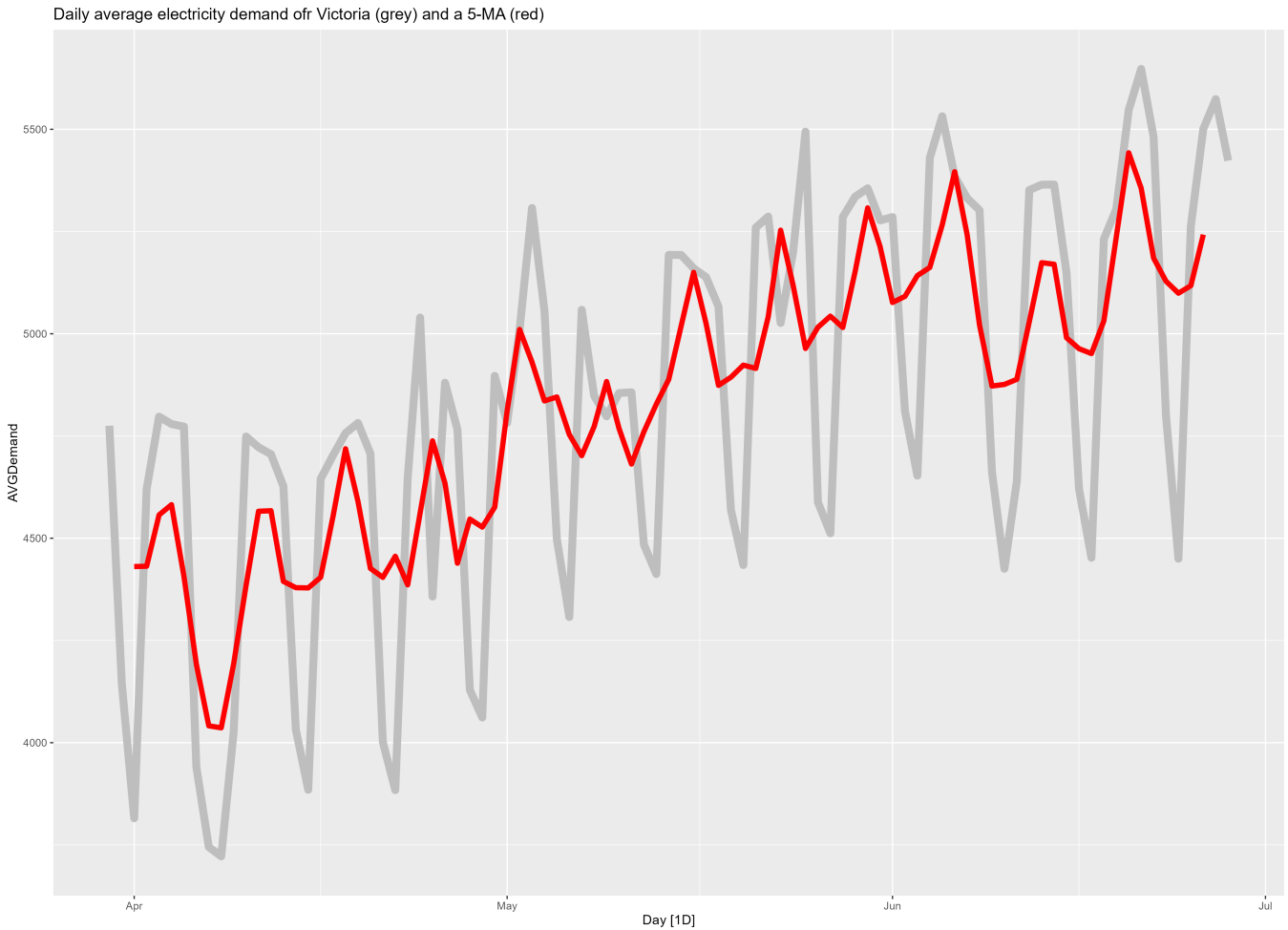
- Original data



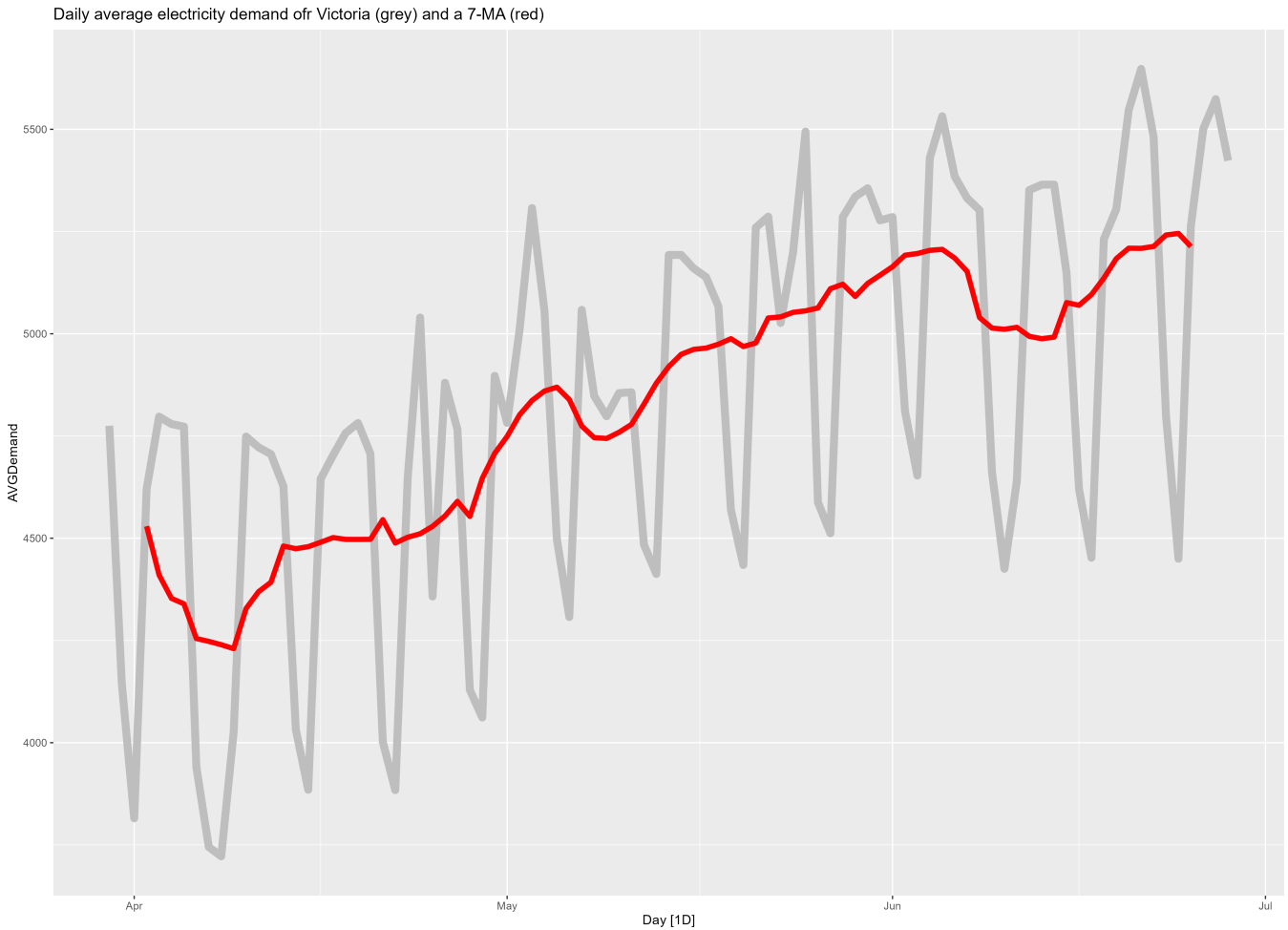
- 3-MA and original data



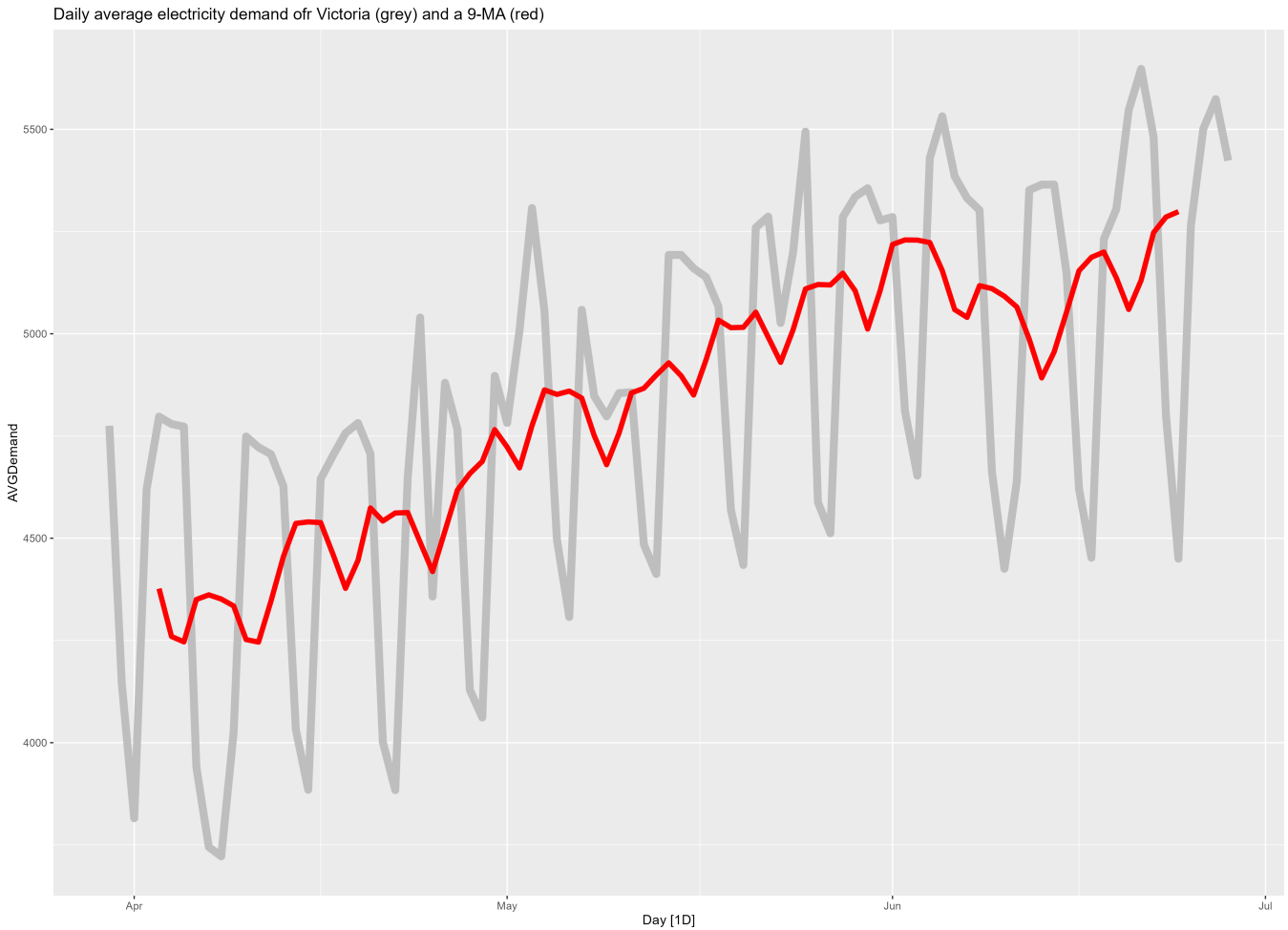
- 5-MA and original data



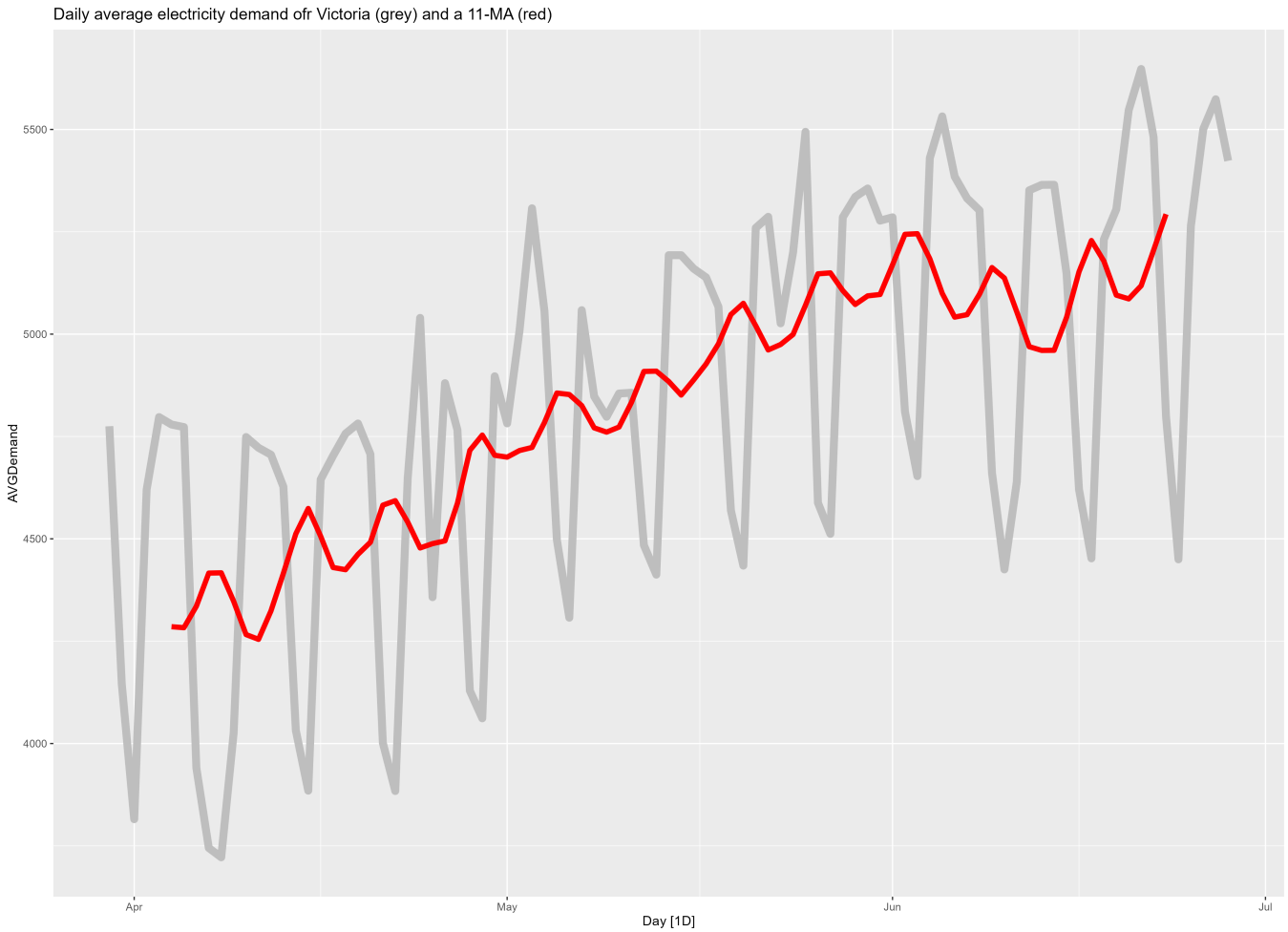
- 7-MA and original data



- 9-MA and original data

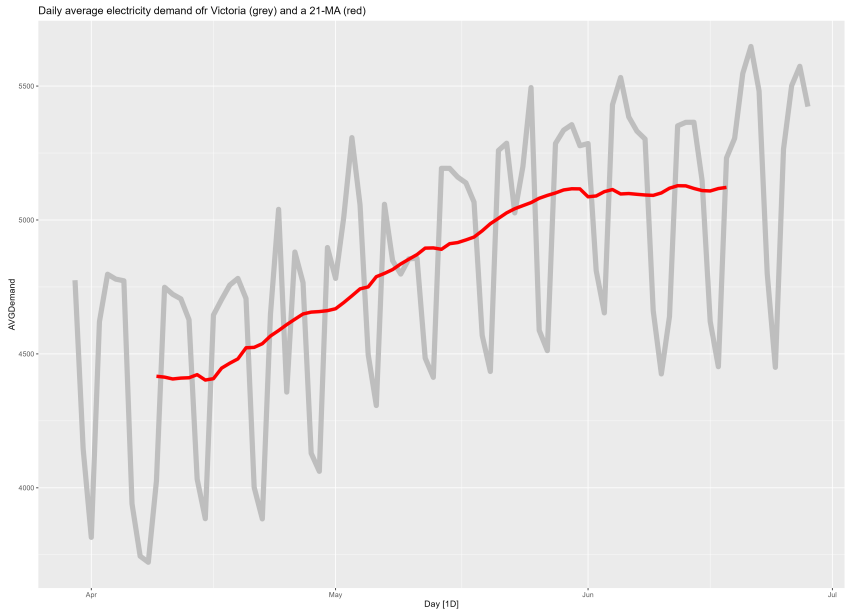


- 11-MA and original data



- Out of the MAs, which one seemed to be the best (smoothest) to you? Do you have any idea why that was the smoothest?

- If the order of the MA is equal to the seasonal length, the MA actually eliminates the effect of seasonality. This is why the 7-MA was looks the smoothest out of them. And if we look at the 21-MA that is even smoother.



- But what do we do if the seasonal length is an even number?

Moving averages of moving averages

- Moving averages of an even order are not symmetric. For example, if you take a 4-MA, you need to decide whether to consider 1 observation before and 2 after your observation, or the other way around.
- However, a moving average of an even order can be made symmetric, if we compute the moving average of the moving average.

- For example, we can compute a *4-MA* and then compute a *2-MA* on the result. In general this is denoted by $2 \times 4 - MA$.
- As an example, consider a simple time series, t_1, t_2, t_3, t_4, etc. In order to compute the *4-MA* we will have (I choose 2 observations before and 1 after):

$$\hat{T}_3 = \frac{1}{4} * (y_1 + y_2 + y_3 + y_4)$$

$$\hat{T}_4 = \frac{1}{4} * (y_2 + y_3 + y_4 + y_5)$$

$$\hat{T}_5 = \frac{1}{4} * (y_3 + y_4 + y_5 + y_6)$$

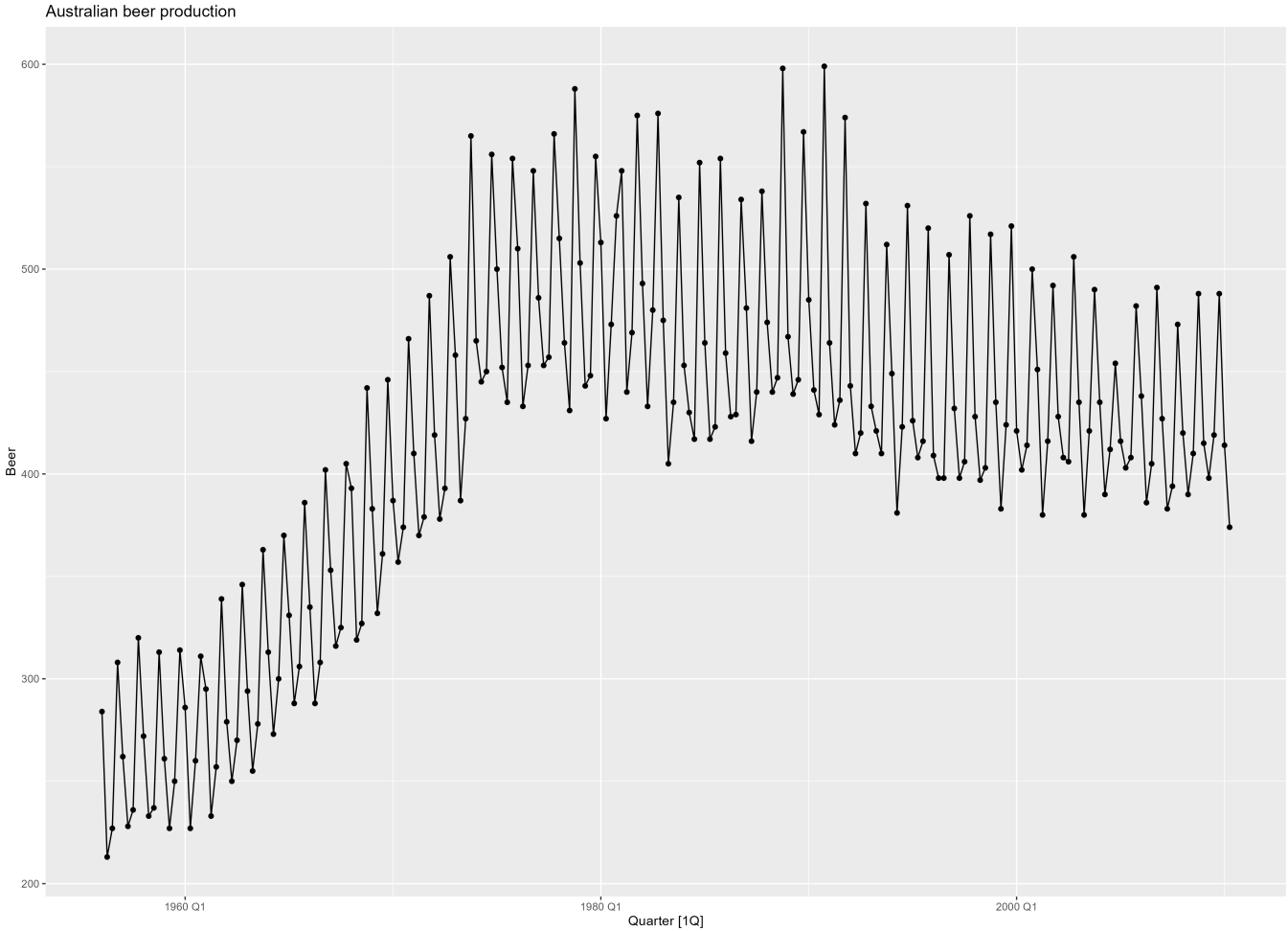
$$\dots$$

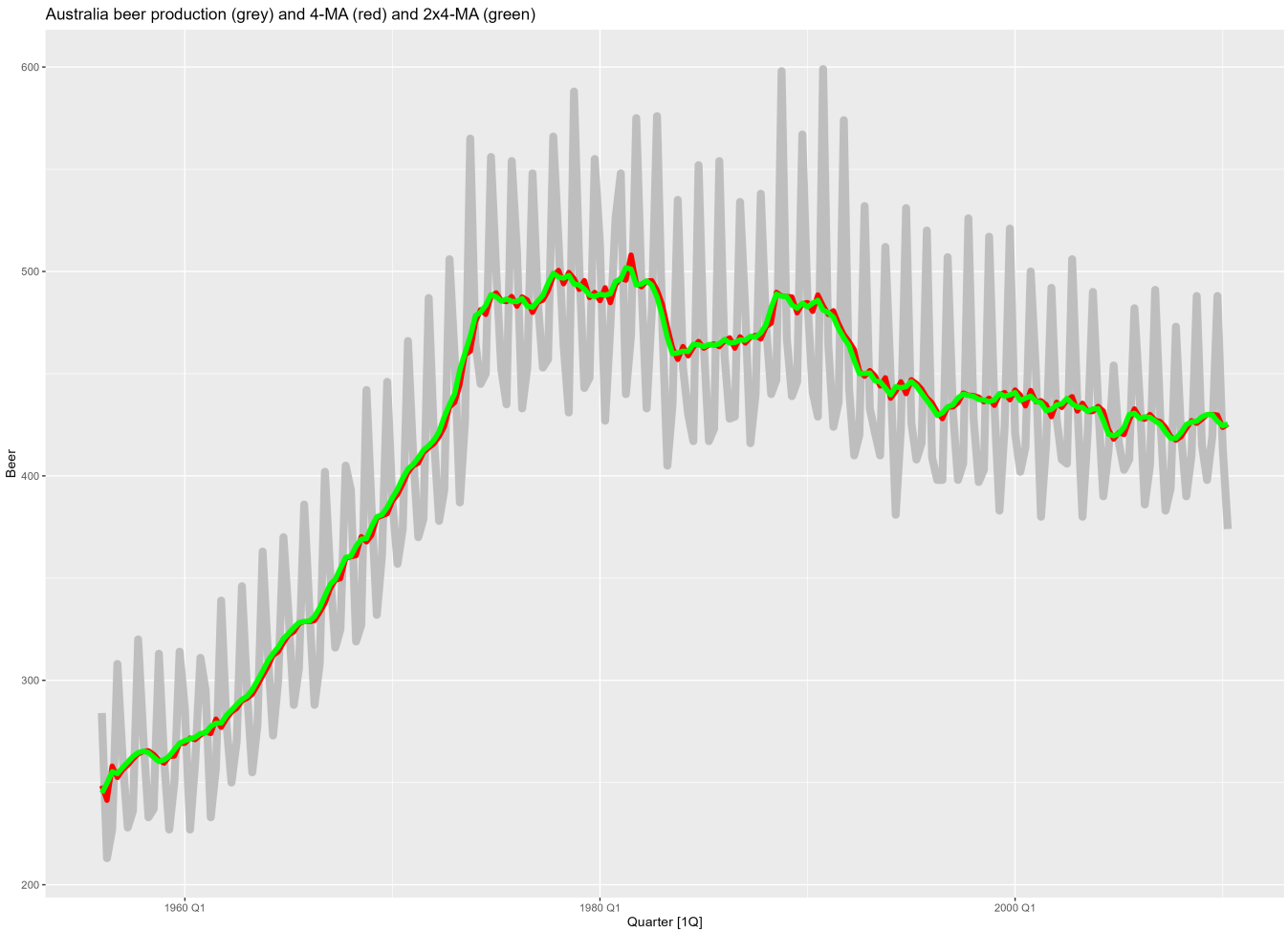
- Then, to compute the *2-MA* on this series, we will have

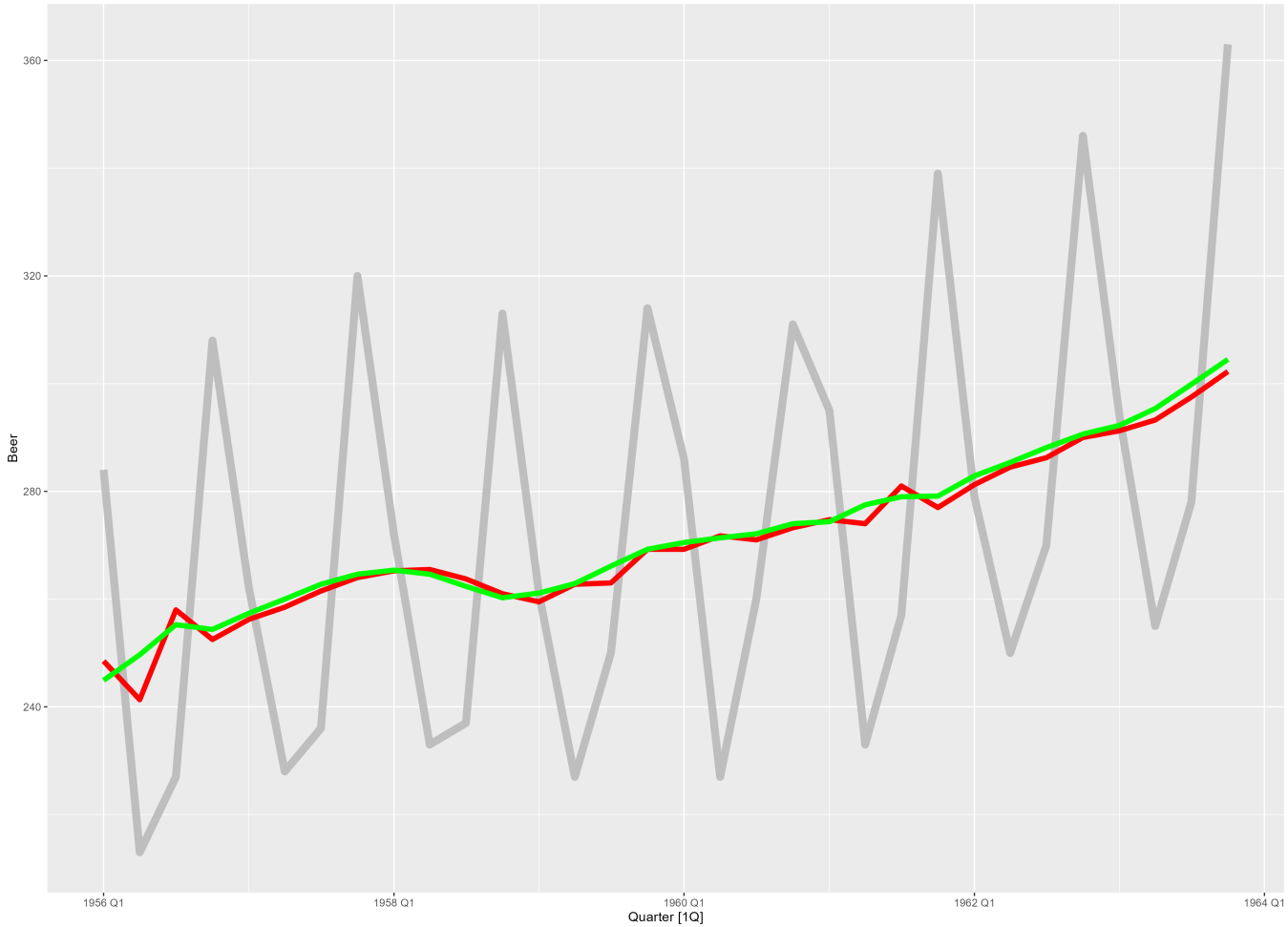
$$\frac{1}{2} * (\hat{T}_3 + \hat{T}_4) = \frac{1}{8}y_1 + \frac{1}{4}y_2 + \frac{1}{4}y_3 + \frac{1}{4}y_4 + \frac{1}{8}y_5$$

- which is a symmetric weighted average of observations.

```
beer |> autoplot() +
  geom_point() +
  labs(title = "Australian beer production")

beer |> mutate(MA4 = slider::slide_dbl(Beer, mean, .before = 2, .after = 1,
      complete = TRUE)) |>
  mutate(MA2x4 = slider::slide_dbl(MA4, mean, .before= 0, .after = 1, complete =
       TRUE)) -> beer_withMA


beer_withMA |>
  autoplot(Beer, color = "grey", size = 3) +
  autolayer(beer_withMA, MA4, color = "red", size = 2) +
  autolayer(beer_withMA, MA2x4, color = "green", size = 2) +
  labs(title = "Australia beer production (grey) and 4-MA (red) and 2x4-MA (
      green)" )

beer_withMA_Short <- beer_withMA |> head(n = 32)
beer_withMA_Short |> autoplot(Beer, color = "grey", size = 3) +
  autolayer(beer_withMA_Short, MA4, color = "red", size = 2) +
  autolayer(beer_withMA_Short, MA2x4, color = "green", size = 2)
```


Australian beer production
600
500
400
300
200
Beer
1960 Q1
1980 Q1
2000 Q1
Quarter [1Q]


Australia beer production (grey) and 4-MA (red) and 2x4-MA (green)
Beer
600
500
400
300
200
1960 Q1
1980 Q1
2000 Q1
Quarter [1Q]


360
320
280
240
Beer
1956 Q1
1958 Q1
1960 Q1
1962 Q1
1964 Q1
Quarter [1Q]

- Different combinations of moving averages are possible, but the general rule is that a moving average of an odd order should be followed by another one of an odd order, while a moving average of an even order should be followed by a moving average of an even order.

- When a moving average of an even order, m is followed by a moving average of order 2, we call that a *centered moving average of order* $m + 1$, where all observations have a weight of $\frac{1}{m}$ except for the first and last which have a weight of $\frac{1}{2m}$.

- Moving averages can be used to estimate the trend-cycle component of a time series. Consider the *2x4-MA* (the general formula derived from the example above):

$$\hat{T}_t = \frac{1}{8}y_{t-2} + \frac{1}{4}y_{t-1} + \frac{1}{4}y_t + \frac{1}{4}y_{t+1} + \frac{1}{8}y_{t+2}$$

- If the data on which we compute the *2x4-MA* is quarterly, then each quarter will have an equal weight in the moving average, since three quarters have a weight of $\frac{1}{4}$ and the two quarters with weight $\frac{1}{8}$ are the same quarter, but in different years. Since every quarter is included with equal weight, the result will average out (smooth out) the seasonal variation. We can get a similar result if the do a *2X8-MA* or *2X12-MA* on quarterly data.

- The trend-cycle component of a time series can be estimated by moving-averages in the following way:
  - If the seasonal period is even and of order m, we use a *2xm-MA*
  - If the seasonal period is odd and of order m, we use a *m-MA*
  - If there are no seasons in the data, just choose an odd value.
- We can choose multiples of m in both cases, but other values will lead to an estimate which probably contains seasonal variation.

Weighted moving average

- Combining moving averages will lead to weighted moving averages.

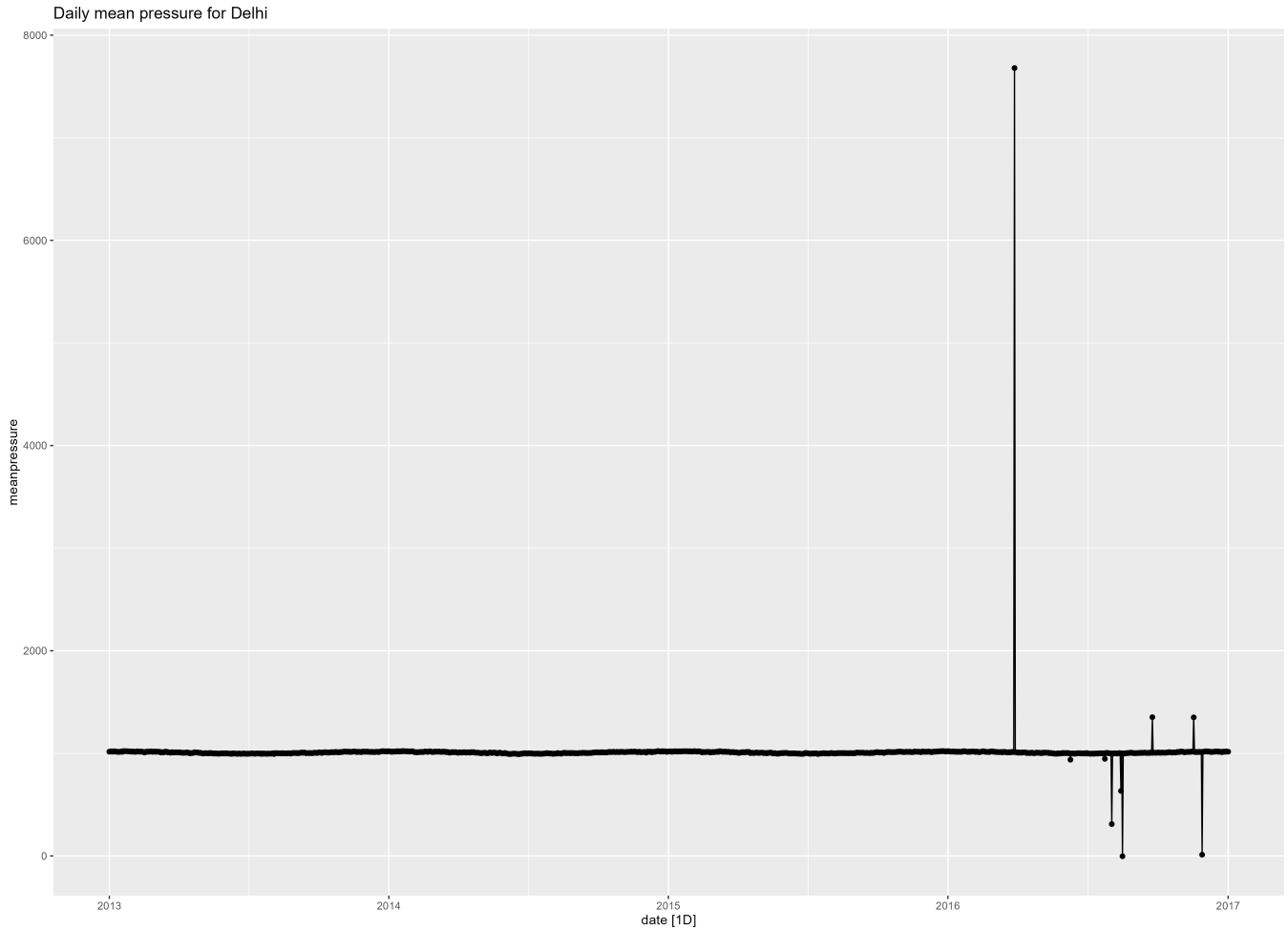
$$\hat{T}_t = \sum_{j=-k}^{k} a_j * y_{t+j}$$

- where $k = \frac{m-1}{2}$, a_j is the weight of observation y_j and it is important that the weights are symmetric and their sum is 1.

- The already mentioned *2x4-MA* will lead to a weighted moving average which is the same as the *5-MA* with the following weights: $[\frac{1}{8}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{8}]$.

- Let us look at another example of computing the moving average.
- We will consider (part of) the Delhi Climate time series.
- It has 1462 daily observations about mean temperature, humidity, wind_speed and mean pressure for Delhi.
- We will use the *meanpressure* time series. Let us visualize it:

```
climate <- read_csv("DailyDelhiClimate.csv")
climate
climate_ts <- climate |> as_tsibble(index = date)
climate_ts |> autoplot(meanpressure) +
  geom_point() +
  labs(title = "Daily mean pressure for Delhi")
```



- What do you observe in the plot?

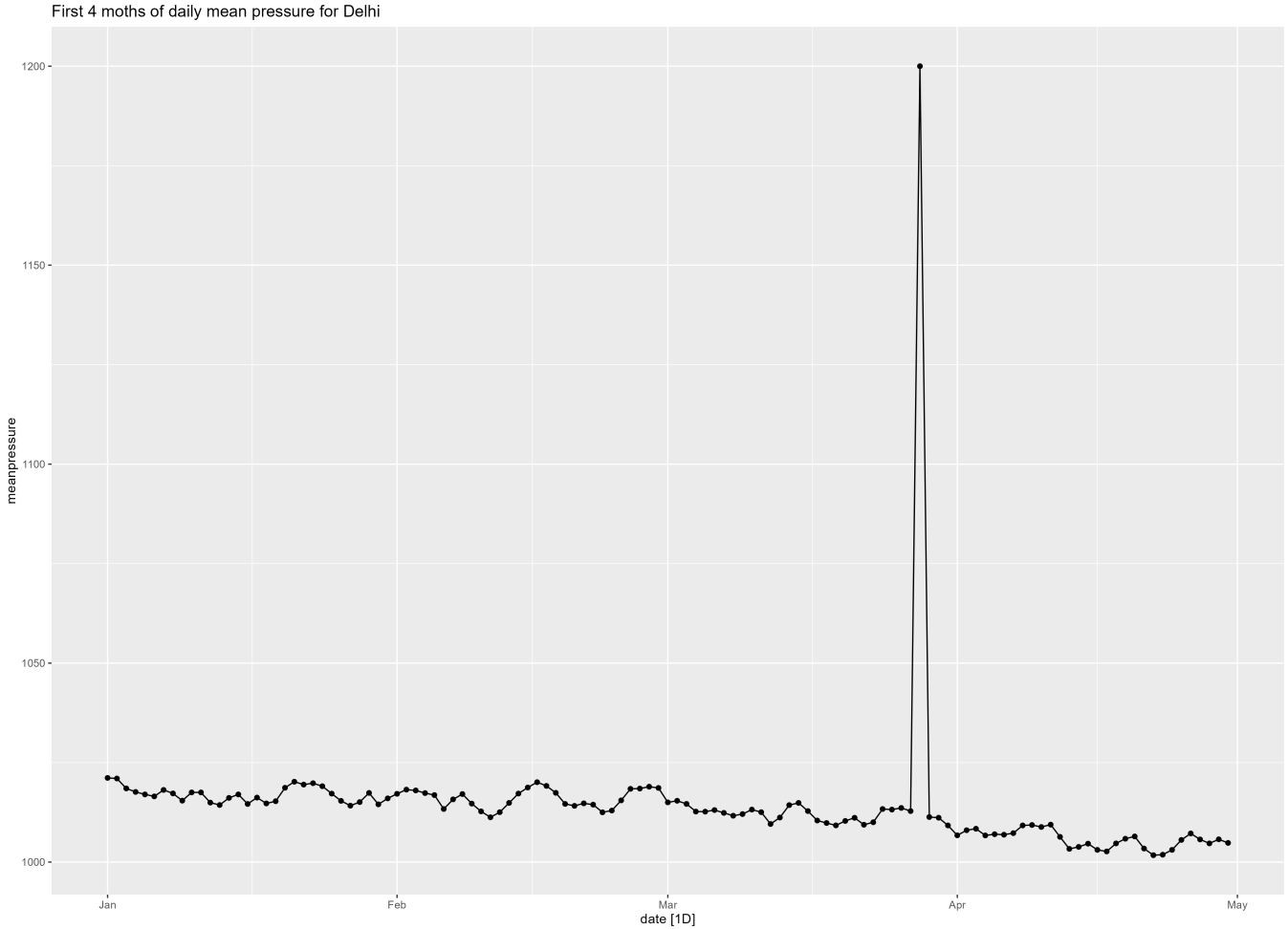
- In the year 2016, the *meanpressure* time series contains several outlier values, the largest one (and chronologically the first one) having a value of over 7000, while most values are around 1000.

- To see that part better, we will do two things:

  - Change the maximum outlier to 1200, to be able to see things better on the plot

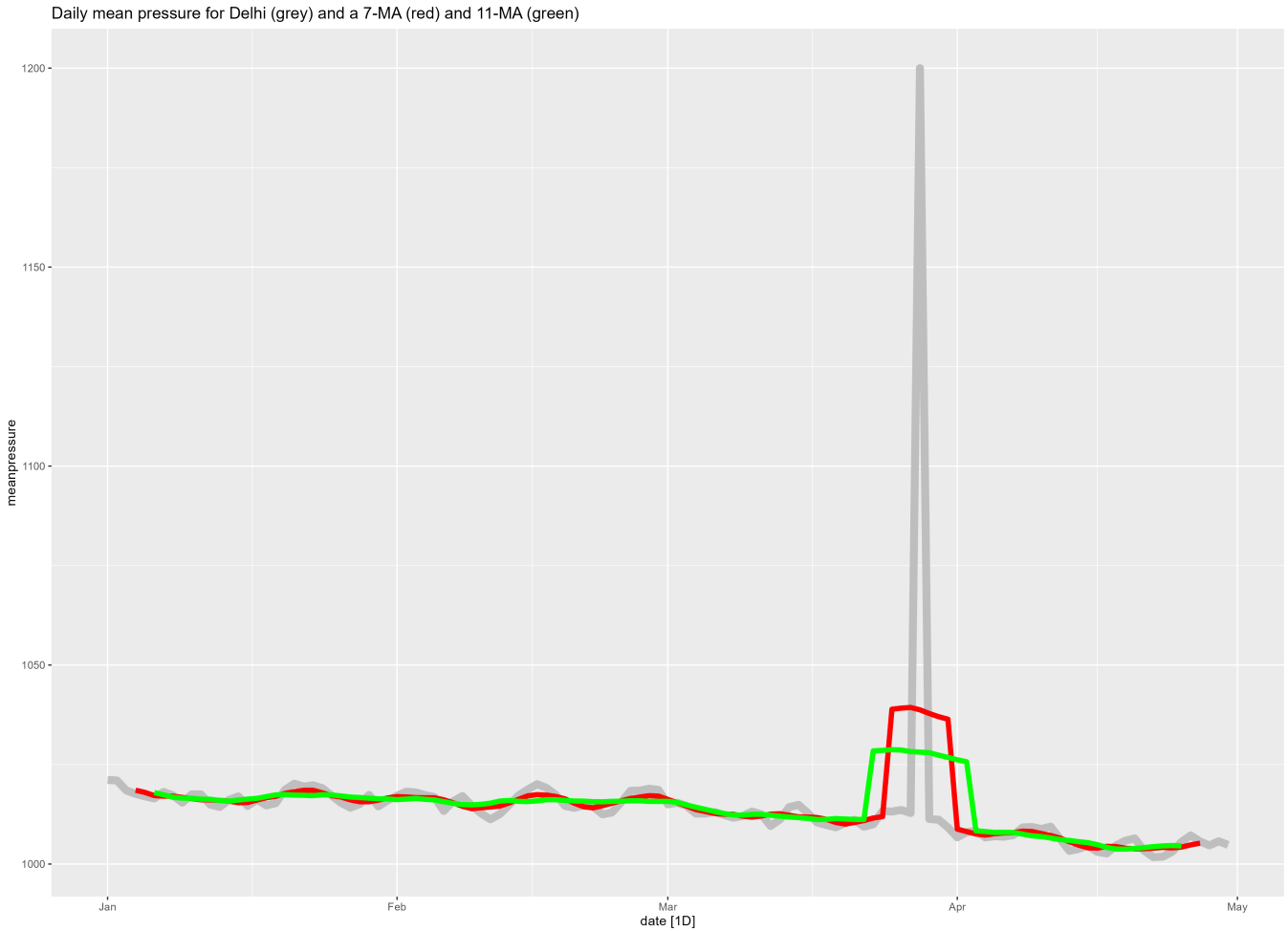
  - Consider only the first 4 months of 2016 (this eliminates the other outliers as well)

```
position <- which(climate_ts$meanpressure == max(climate_ts$meanpressure))
position

climate_ts$meanpressure[1183] <- 1200

climate_ts_short <- climate_ts |> filter(year(date) == 2016, month(date) < 5)

climate_ts_short |> autoplot(meanpressure) +
  geom_point() +
  labs(title = "First 4 moths of daily mean pressure for Delhi")
```


First 4 moths of daily mean pressure for Delhi
meanpressure
1200
1150
1100
1050
1000
Jan
Feb
Mar
Apr
May
date [1D]

- The data does not have seasonalities (can be checked with seasonal subseries plot) so we will compute a 7-MA and an 11-MA on it and plot them together with the original data.

```
climate_ts_short <- climate_ts_short |>
  mutate(MA7 = slider::slide_dbl(meanpressure, mean, .before = 3, .after = 3, .
      complete = TRUE)) |>
  mutate(MA11 = slider::slide_dbl(meanpressure, mean, .before = 5, .after = 5, .
      complete = TRUE))

climate_ts_short |> autoplot(meanpressure, color = "grey", size = 3) +
  autolayer(climate_ts_short, MA7, color = "red", size = 2) +
  autolayer(climate_ts_short, MA11, color = "green", size = 2)
```



- What do you notice on the plots?

Moving median

- The effect of the outlier is visible on the moving averages, which becomes significantly higher around it.
- For a higher value of m, the effect is reduced, but it is still significantly modified compared to the others.
- For such cases with an outlier value (which might actually be a real value), a *moving median* might be a better option.
- In case of an odd m, in order to determine the median of the m values, sort them, and pick the middle one.
- In the *slider_dbl* function the moving median can be computed (instead of the moving average) if the second parameter is *median* instead of *mean*.

```
climate_ts_short <- climate_ts_short |>
  mutate(MM7 = slider::slide_dbl(meanpressure, median, .before = 3, .after = 3,
     .complete = TRUE))
```

- Moving average of order 7 (red line) vs. moving median of order 7 (green line)